

Binary Trees

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

July 22, 2026

Contents

1	Binary Trees	1
1.1	The Shape of Every Tree Problem	1
1.2	Recursive Traversals: Preorder, Inorder, Postorder	3
1.3	Level Order Traversal (BFS)	5
1.4	Iterative Preorder Traversal	7
1.5	Iterative Inorder Traversal	9
1.6	Iterative Postorder Traversal (Two Stacks)	10
1.7	Iterative Postorder Traversal (One Stack)	12
1.8	Preorder, Inorder, and Postorder in One Traversal	14
1.9	Maximum Depth (Height) of a Binary Tree	16
1.10	Check for a Balanced Binary Tree	18
1.11	Diameter of a Binary Tree	20
1.12	Maximum Path Sum	22
1.13	Check if Two Trees Are Identical	23
1.14	Zigzag (Spiral) Level Order Traversal	25
1.15	Boundary Traversal of a Binary Tree	27
1.16	Vertical Order Traversal	30
1.17	Top View of a Binary Tree	31
1.18	Bottom View of a Binary Tree	33
1.19	Left View and Right View of a Binary Tree	35
1.20	Symmetric Binary Tree	37
1.21	Root-to-Node Path	39
1.22	Lowest Common Ancestor in a Binary Tree	41
1.23	Maximum Width of a Binary Tree	42
1.24	Children Sum Property	44
1.25	All Nodes at Distance K from a Target	46
1.26	Minimum Time to Burn the Tree	48
1.27	Count Nodes in a Complete Binary Tree	51
1.28	Construct a Binary Tree from Preorder and Inorder	53
1.29	Construct a Binary Tree from Postorder and Inorder	55
1.30	Serialize and Deserialize a Binary Tree	57
1.31	Morris Inorder Traversal	59
1.32	Morris Preorder Traversal	61
1.33	Flatten a Binary Tree to a Linked List	63

Chapter 1

Binary Trees

A binary tree is recursion made visible. Almost every problem in this chapter has the same shape underneath: solve the left subtree, solve the right subtree, then combine those two answers into an answer for the current node. Master that single “ask both children, then decide” rhythm and the entire chapter – traversals, heights, diameters, views, path sums, reconstruction – collapses into variations on one idea.

1.1 The Shape of Every Tree Problem

A **binary tree** is either empty, or a **node** holding a value together with two children, each itself the root of a binary tree. That recursive definition is not just a formality – it is the template for almost every algorithm here. A node knows nothing about the tree except its own value and two pointers; any global fact (the height, the diameter, whether the tree is balanced, the maximum path sum) must be assembled by asking each child for its answer and combining.

The One Rhythm Behind This Whole Chapter

For a huge fraction of tree problems the skeleton is identical:

$$\text{solve}(\text{node}) = \text{combine}(\text{solve}(\text{node.left}), \text{solve}(\text{node.right}), \text{node.val}).$$

The only thing that changes from problem to problem is *what each call returns* and *how the two child answers are combined*. Height returns $1 + \max$; “is balanced” returns height but also checks a condition; diameter returns height but also updates a global best; max-path-sum returns the best downward path but updates a global best using both sides. Learn to see every problem as “what do I return, and how do I combine,” and the code writes itself.

How to Recognise Which Technique a Tree Problem Needs

Recurring Binary-Tree Keywords

- “**Visit/print every node in some order**” – a **traversal**. Depth-first (preorder, inorder, postorder) is plain recursion, or an explicit stack when an interviewer forbids recursion; breadth-first (level order) is a **queue**.
- “**Height, depth, balanced, diameter, maximum path sum**” – **bottom-up (postorder) recursion**: each node’s answer needs both children’s answers *first*, so compute children, then the node. A single global variable often records the best answer while each call returns the value its parent needs.
- “**Top view, bottom view, vertical order, width**” – **level-order (BFS) with coordinates**: tag each node with a horizontal distance (and sometimes depth), then read the map off in the required order.
- “**Left view, right view, level order, zigzag**” – **BFS level by level**, picking the

first/last node of each level or alternating direction.

- **“Lowest common ancestor, root-to-node path, nodes at distance K, minimum time to burn” – path/relationship problems:** either a single recursion that reports back up whether a target was found, or a preprocessing pass that records each node’s parent so the tree can be walked in *any* direction.
- **“Construct a tree from its traversals, serialize/deserialize” – reconstruction:** one traversal fixes the root, another (or a delimiter scheme) splits the remainder into left and right, applied recursively.
- **“ $O(1)$ space traversal, no stack, no recursion” – Morris traversal,** which borrows the tree’s own null right pointers as temporary threads back to ancestors.

Throughout this chapter a node is the following structure, and every code listing is a complete, compilable program:

```
1 struct TreeNode {  
2     int val;  
3     TreeNode* left;  
4     TreeNode* right;  
5     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}  
6 };
```

Each section follows the established structure: problem statement and keywords; the reasoning that identifies the right traversal or recursive technique and why it is correct; the algorithm in words and pseudocode; a hand-traced dry run; complete compilable C++ code; and a time/space complexity analysis.

1.2 Recursive Traversals: Preorder, Inorder, Postorder

Problem Statement. Given the root of a binary tree, return its preorder, inorder, and postorder traversals. Preorder visits *node, left, right*; inorder visits *left, node, right*; postorder visits *left, right, node*.

Pattern Recognition

“Visit every node in a fixed relative order” is the definition of a depth-first traversal. The only thing separating the three is **where the current node is printed relative to its two recursive calls** – before both (pre), between them (in), or after both (post). If you can write one, you can write all three by moving a single line.

Understanding the problem carefully

The recursive structure of a binary tree hands you the traversal almost for free. To process a whole tree you process its root and its two subtrees; each subtree is processed the same way. The word “pre/in/post” refers only to *when the root itself is recorded* relative to recursing into the left and right subtrees.

One Template, Three Positions

Every depth-first traversal is the same three actions – RECORD NODE, RECURSE LEFT, RECURSE RIGHT – performed in a different order:

- **Preorder:** record, left, right.
- **Inorder:** left, record, right.
- **Postorder:** left, right, record.

The recursion itself is byte-for-byte identical; only the placement of the “record” step moves.

Algorithm

1. INORDER(*node, out*):
 - (a) If *node* is null, return.
 - (b) INORDER(*node.left, out*).
 - (c) Append *node.val* to *out*.
 - (d) INORDER(*node.right, out*).
2. Preorder and postorder are the same three lines with the append moved to the top or the bottom respectively.

Dry run

Dry Run. Tree: root 1, right child 2, and 2’s left child 3.

- **Preorder:** record 1; left of 1 is null; go right to 2; record 2; left of 2 is 3; record 3 (its children null); right of 2 null. Result: 1, 2, 3.
- **Inorder:** left of 1 null; record 1; go right to 2; left of 2 is 3, record 3; record 2; right null. Result: 1, 3, 2.
- **Postorder:** left of 1 null; right subtree of 1: left of 2 is 3, record 3; right of 2 null; record 2; record 1. Result: 3, 2, 1.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 void preorder(TreeNode* node, vector<int>& out) {
12     if (node == nullptr) return;
13     out.push_back(node->val);    // record
14     preorder(node->left, out);
15     preorder(node->right, out);
16 }
17
18 void inorder(TreeNode* node, vector<int>& out) {
19     if (node == nullptr) return;
20     inorder(node->left, out);
21     out.push_back(node->val);    // record
22     inorder(node->right, out);
23 }
24
25 void postorder(TreeNode* node, vector<int>& out) {
26     if (node == nullptr) return;
27     postorder(node->left, out);
28     postorder(node->right, out);
29     out.push_back(node->val);    // record
30 }
31
32 int main() {
33     TreeNode* root = new TreeNode(1);
34     root->right = new TreeNode(2);
35     root->right->left = new TreeNode(3);
36
37     vector<int> pre, in, post;
38     preorder(root, pre);
39     inorder(root, in);
40     postorder(root, post);
41
42     auto printv = [](const string& name, vector<int>& v) {
43         cout << name << ": ";
44         for (int x : v) cout << x << " ";
45         cout << "\n";
46     };
47     printv("preorder ", pre);
48     printv("inorder  ", in);
49     printv("postorder", post);
50     return 0;
51 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is visited exactly once. **Space Complexity.** $O(h)$ for the recursion stack, where h is the tree’s height ($O(n)$ in the worst case of a skewed tree, $O(\log n)$ when balanced), plus $O(n)$ for the output array.

Key Takeaway

The three depth-first traversals are one algorithm with the “record” step in three different positions. Every later depth-first problem in this chapter is a traversal with extra work grafted onto that record step, so internalise this skeleton before moving on.

1.3 Level Order Traversal (BFS)

Problem Statement. Given the root of a binary tree, return its level order traversal: the node values grouped level by level, from the root’s level downward, left to right within each level.

Pattern Recognition

“Level by level,” “breadth-first,” “top-to-bottom” are the signature of a **queue**-based traversal. Whenever a tree problem asks for anything organised by *depth* – level order, left/right view, bottom/top view, maximum width, minimum time to burn – a BFS with a queue is almost always the backbone.

Understanding the problem carefully

Depth-first recursion dives all the way down one branch before touching the next, so it visits nodes out of level order. To visit nodes in order of increasing depth we need the opposite discipline: fully process one level before starting the next. A **FIFO queue** does exactly this – if we always enqueue a node’s children when we dequeue it, then all of level k is enqueued before any node of level $k + 1$, so the queue hands nodes back in level order automatically.

Freeze the Level Size Before Draining It

When you begin processing a level, the queue holds *exactly* the nodes of that level and nothing else. Record its size s first, then dequeue precisely s nodes: those s are the current level, and the children you enqueue while draining them form the next level, cleanly separated. This “read the size, then loop that many times” trick is what lets a single queue produce grouped-by-level output.

Algorithm

1. If root is null, return empty.
2. Push root into a queue.
3. While the queue is non-empty:
 - (a) Let s be the current queue size; start a new level list.
 - (b) Repeat s times: pop the front node, append its value to the level list, and push its non-null children.
 - (c) Append the level list to the answer.

Dry run

Dry Run. Tree: root 1; children 2 (left) and 3 (right); 2's children 4, 5.

- Queue = [1], size 1: pop 1, push 2, 3. Level = [1].
- Queue = [2, 3], size 2: pop 2 (push 4, 5), pop 3. Level = [2, 3].
- Queue = [4, 5], size 2: pop 4, pop 5. Level = [4, 5].
- Result: [[1], [2, 3], [4, 5]].

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 vector<vector<int>> levelOrder(TreeNode* root) {
12     vector<vector<int>> ans;
13     if (root == nullptr) return ans;
14
15     queue<TreeNode*> q;
16     q.push(root);
17     while (!q.empty()) {
18         int s = q.size();
19         vector<int> level;
20         for (int i = 0; i < s; i++) {
21             TreeNode* node = q.front();
22             q.pop();
23             level.push_back(node->val);
24             if (node->left) q.push(node->left);
25             if (node->right) q.push(node->right);
26         }
27         ans.push_back(level);
28     }
29     return ans;
30 }
31
32 int main() {
33     TreeNode* root = new TreeNode(1);
34     root->left = new TreeNode(2);
35     root->right = new TreeNode(3);
36     root->left->left = new TreeNode(4);
37     root->left->right = new TreeNode(5);
38
39     for (auto& level : levelOrder(root)) {
40         for (int x : level) cout << x << " ";
41         cout << "\n";
42     }
43     return 0;
44 }

```

Complexity Analysis

Time Complexity. $O(n)$ – every node is enqueued and dequeued exactly once. **Space Complexity.** $O(n)$ for the queue, which in the worst case (a full last level) holds up to $n/2$ nodes at once.

Key Takeaway

The queue plus the “freeze the level size” loop is the single reusable engine behind every by-depth view in this chapter. Get it fluent here; top view, bottom view, right view and maximum width all reuse it verbatim with a small tweak to what gets recorded.

1.4 Iterative Preorder Traversal

Problem Statement. Return the preorder traversal (node, left, right) of a binary tree *without* recursion, using an explicit stack.

Pattern Recognition

“*Do this traversal iteratively / without recursion*” means: simulate the call stack yourself with an explicit **stack**. Preorder is the easiest of the three to convert because the node is recorded the instant it is seen, before either child – so there is no “come back to this node later” bookkeeping to manage.

Understanding the problem carefully

Recursion works by an implicit stack of pending calls. To remove the recursion we make that stack explicit. In preorder we want to output a node, then fully handle its left subtree, then its right subtree. A stack (LIFO) lets us achieve exactly this if we are careful about push order.

Push Right Before Left So Left Pops First

Pop a node and immediately record it. Then push its *right* child and *then* its *left* child. Because a stack returns the most recently pushed item first, the left child is processed before the right – matching preorder’s “node, then all of left, then all of right.” Pushing right first is the whole trick; reverse it and you would get “node, right, left.”

Algorithm

1. If root is null, return empty. Push root onto a stack.
2. While the stack is non-empty:
 - (a) Pop a node; append its value to the output.
 - (b) If it has a right child, push it.
 - (c) If it has a left child, push it.

Dry run

Dry Run. Tree: root 1; children 2 (left), 3 (right); 2’s left child 4.

- Stack = [1]: pop 1, record 1, push 3 then 2. Stack = [3, 2] (top is 2).
- Pop 2, record 2, push (no right), push 4. Stack = [3, 4].

- Pop 4, record 4 (no children). Stack = [3].
- Pop 3, record 3. Stack empty.
- Result: 1, 2, 4, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 vector<int> preorderIterative(TreeNode* root) {
12     vector<int> out;
13     if (root == nullptr) return out;
14
15     stack<TreeNode*> st;
16     st.push(root);
17     while (!st.empty()) {
18         TreeNode* node = st.top();
19         st.pop();
20         out.push_back(node->val);
21         if (node->right) st.push(node->right); // right first
22         if (node->left) st.push(node->left);   // so left pops first
23     }
24     return out;
25 }
26
27 int main() {
28     TreeNode* root = new TreeNode(1);
29     root->left = new TreeNode(2);
30     root->right = new TreeNode(3);
31     root->left->left = new TreeNode(4);
32
33     for (int x : preorderIterative(root)) cout << x << " ";
34     cout << "\n";
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is pushed and popped exactly once. **Space Complexity.** $O(h)$ for the stack in the worst case (the stack never holds more than one node per level of the current path plus pending right children).

Key Takeaway

Preorder is the friendliest traversal to make iterative because “record” happens on first sight. The push-right-then-left order is the reusable detail. Inorder and postorder are harder precisely because their record step is delayed until after one or both subtrees are done.

1.5 Iterative Inorder Traversal

Problem Statement. Return the inorder traversal (left, node, right) of a binary tree without recursion, using an explicit stack.

Pattern Recognition

Inorder's record step is *delayed*: a node cannot be output until its entire left subtree is done. So the iterative version has two phases that alternate – **dive left pushing everything**, then **pop, record, and turn right**. This “go as far left as possible, then process, then step right” loop is worth memorising; it reappears in the BST iterator.

Understanding the problem carefully

In inorder we must reach the leftmost node before recording anything. The stack stores the chain of ancestors we still owe a visit to. We push nodes as we descend left; when we can descend no further, the top of the stack is the next node to record; after recording it we move to its right subtree and repeat the leftward dive from there.

A Pointer and a Stack, Cooperating

Keep a running pointer *curr*. While *curr* is non-null, push it and move *curr* to its left child (the leftward dive). When *curr* becomes null, pop a node, record it, and set *curr* to that node's right child (turn right, then dive left again). The loop ends only when both the stack is empty *and* *curr* is null.

Algorithm

1. Set *curr* = root; use an empty stack.
2. While *curr* is non-null **or** the stack is non-empty:
 - (a) While *curr* is non-null: push *curr*, set *curr* = *curr*.left.
 - (b) Pop a node; append its value; set *curr* = node.right.

Dry run

Dry Run. Tree: root 2; left child 1; right child 3.

- *curr*= 2: push 2, *curr*= 1. Push 1, *curr*=null.
- Pop 1, record 1, *curr*=null (right of 1).
- Pop 2, record 2, *curr*= 3.
- Push 3, *curr*=null. Pop 3, record 3.
- Result: 1, 2, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;

```

```

6   TreeNode* left;
7   TreeNode* right;
8   TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 vector<int> inorderIterative(TreeNode* root) {
12     vector<int> out;
13     stack<TreeNode*> st;
14     TreeNode* curr = root;
15
16     while (curr != nullptr || !st.empty()) {
17         while (curr != nullptr) { // dive left
18             st.push(curr);
19             curr = curr->left;
20         }
21         curr = st.top();           // leftmost pending node
22         st.pop();
23         out.push_back(curr->val); // record
24         curr = curr->right;        // turn right
25     }
26     return out;
27 }
28
29 int main() {
30     TreeNode* root = new TreeNode(2);
31     root->left = new TreeNode(1);
32     root->right = new TreeNode(3);
33
34     for (int x : inorderIterative(root)) cout << x << " ";
35     cout << "\n";
36     return 0;
37 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is pushed once and popped once. **Space Complexity.** $O(h)$ – the stack holds at most one node per level along the current leftmost chain.

Key Takeaway

The “dive left, pop-record, turn right” loop is the canonical iterative inorder and the exact engine behind a BST iterator that returns sorted values one at a time. Memorise the two-part loop shape; the same structure handles “next smallest” queries on demand.

1.6 Iterative Postorder Traversal (Two Stacks)

Problem Statement. Return the postorder traversal (left, right, node) of a binary tree without recursion, using two stacks.

Pattern Recognition

Postorder is the hardest to make iterative, because a node is recorded only *after both* subtrees. The two-stack method sidesteps the difficulty with a slick observation: postorder is the exact **reverse** of a slightly modified preorder. If you can produce “node, right, left” and then reverse it, you have postorder for free.

Understanding the problem carefully

Postorder is left, right, node. Read it backwards and you get node, right, left – which is just preorder with the two child visits swapped. So if we run a preorder-like traversal that pushes *left before right* (giving node, right, left order) and collect the results, then reverse the whole collection, we recover true postorder. The second stack is simply the container we reverse.

Postorder = reverse of (node, right, left)

Run a preorder variant using stack 1 that emits nodes in *node, right, left* order, pushing each emitted node onto stack 2. Popping stack 2 at the end yields those nodes in reverse – i.e. *left, right, node*, exactly postorder. No delayed-visit bookkeeping is needed.

Algorithm

1. If root is null, return empty. Push root onto stack 1.
2. While stack 1 is non-empty:
 - (a) Pop a node from stack 1 and push it onto stack 2.
 - (b) Push that node's *left* child, then its *right* child, onto stack 1 (if they exist).
3. Pop everything off stack 2 into the output; that is the postorder.

Dry run

Dry Run. Tree: root 1; children 2 (left), 3 (right).

- Stack1 = [1]: pop 1 → stack2 = [1]; push 2 then 3. Stack1 = [2, 3].
- Pop 3 → stack2 = [1, 3] (no children). Stack1 = [2].
- Pop 2 → stack2 = [1, 3, 2]. Stack1 empty.
- Empty stack2: 2, 3, 1. That is the postorder.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 vector<int> postorderTwoStacks(TreeNode* root) {
12     vector<int> out;
13     if (root == nullptr) return out;
14
15     stack<TreeNode*> s1, s2;
16     s1.push(root);
17     while (!s1.empty()) {
18         TreeNode* node = s1.top();
19         s1.pop();

```

```

20     s2.push(node);
21     if (node->left) s1.push(node->left);
22     if (node->right) s1.push(node->right);
23 }
24 while (!s2.empty()) {
25     out.push_back(s2.top()->val);
26     s2.pop();
27 }
28 return out;
29 }
30
31 int main() {
32     TreeNode* root = new TreeNode(1);
33     root->left = new TreeNode(2);
34     root->right = new TreeNode(3);
35
36     for (int x : postorderTwoStacks(root)) cout << x << " ";
37     cout << "\n";
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(n)$ – every node passes through both stacks once. **Space Complexity.** $O(n)$ – stack 2 accumulates all n nodes before the final reversal, which is the price of the simpler logic.

Key Takeaway

Recognising that postorder is a reversed, child-swapped preorder turns a fiddly problem into two easy passes. The trade is memory: it holds all n nodes at once. When an interviewer demands $O(h)$ space, use the one-stack method in the next section instead.

1.7 Iterative Postorder Traversal (One Stack)

Problem Statement. Return the postorder traversal of a binary tree without recursion, using a single stack and $O(h)$ auxiliary space.

Pattern Recognition

The tell-tale that a one-stack postorder is wanted is a space constraint: the two-stack method is simpler but stores all n nodes. With one stack we must handle the genuine difficulty head-on – *a node may sit on the stack while we visit first its left subtree and only later its right* – so we need a way to know, when a node resurfaces, whether we are returning from its left or its right side.

Understanding the problem carefully

In postorder we record a node only after *both* its children are done. When we pop back up to a node, we must distinguish “I have finished its left subtree, now go right” from “I have finished its right subtree, now record it.” Tracking the **last node we recorded** resolves this ambiguity: if that last-recorded node is the current node’s right child (or the right child is null), both subtrees are complete and the node may be recorded; otherwise there is still a right subtree to descend into.

Remember the Last Node You Output

Keep *curr* (the cursor) and *lastVisited* (the most recently recorded node). Descend left, pushing. At the top of the stack: if its right child exists and has not just been visited, move right; otherwise record the top, pop it, and set *lastVisited* to it. That single comparison against *lastVisited* tells you which side you are returning from.

Algorithm

1. *curr* = root; *lastVisited* = null; empty stack.
2. While *curr* is non-null or the stack is non-empty:
 - (a) While *curr* is non-null: push *curr*, *curr* = *curr*.left.
 - (b) Let *peek* = top of stack. If *peek*.right exists and *peek*.right $\neq$ *lastVisited*: set *curr* = *peek*.right.
 - (c) Else: record *peek*, pop it, set *lastVisited* = *peek*.

Dry run

Dry Run. Tree: root 1; left 2; right 3.

- Dive left from 1: push 1, push 2. *curr*=null.
- Peek 2: right is null $\Rightarrow$ record 2, pop, *lastVisited*= 2.
- Peek 1: right 3 $\neq$ *lastVisited* $\Rightarrow$ *curr*= 3. Push 3.
- Peek 3: right null $\Rightarrow$ record 3, pop, *lastVisited*= 3.
- Peek 1: right 3 = *lastVisited* $\Rightarrow$ record 1, pop.
- Result: 2, 3, 1.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 vector<int> postorderOneStack(TreeNode* root) {
12     vector<int> out;
13     stack<TreeNode*> st;
14     TreeNode* curr = root;
15     TreeNode* lastVisited = nullptr;
16
17     while (curr != nullptr || !st.empty()) {
18         while (curr != nullptr) {
19             st.push(curr);
20             curr = curr->left;

```

```

21     }
22     TreeNode* peek = st.top();
23     if (peek->right != nullptr && peek->right != lastVisited) {
24         curr = peek->right;           // still owe the right
25         subtree
26     } else {
27         out.push_back(peek->val);      // both sides done: record
28         lastVisited = peek;
29         st.pop();
30     }
31     return out;
32 }
33
34 int main() {
35     TreeNode* root = new TreeNode(1);
36     root->left = new TreeNode(2);
37     root->right = new TreeNode(3);
38
39     for (int x : postorderOneStack(root)) cout << x << " ";
40     cout << "\n";
41     return 0;
42 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is pushed once, and the peek/right check amortises to $O(1)$ per node. **Space Complexity.** $O(h)$ – the stack only ever holds the current root-to-node path, unlike the two-stack version’s $O(n)$.

Key Takeaway

When you must return from a node’s left versus right side and act differently, a *lastVisited* pointer is the standard way to tell them apart without extra per-node flags. This “remember where you came from” idea generalises to any iterative post-processing traversal.

1.8 Preorder, Inorder, and Postorder in One Traversal

Problem Statement. Compute all three depth-first traversals – preorder, inorder, and postorder – in a *single* pass over the tree, using one explicit stack.

Pattern Recognition

“All three in one traversal” is a state-machine problem. Each node is genuinely visited three times during a depth-first walk: on the way down (before its left subtree), between its subtrees, and on the way up (after its right subtree). Attaching a **state counter** 1/2/3 to each stacked node lets one loop record the node into the right list at the right moment.

Understanding the problem carefully

Picture the classic “draw a line around the tree” visualisation of a DFS: your pen touches each node three times – once on its left, once underneath, once on its right. Preorder records the first touch, inorder the second, postorder the third. If we stack each node together with a counter saying which touch is next, a single loop can dispatch it to the correct output list and advance its counter.

One Node, Three Touches, Three States

Push each node with state 1. When popped:

- **State 1** (first touch): add to *preorder*; increment its state to 2 and re-push it; then push its left child (state 1).
- **State 2** (second touch): add to *inorder*; increment to 3 and re-push; then push its right child (state 1).
- **State 3** (third touch): add to *postorder*; discard the node.

Algorithm

1. Push (root, 1) onto a stack. While non-empty, pop (*node*, *state*):
 - (a) If *state* = 1: append to *preorder*; push (*node*, 2); if left child exists push (left, 1).
 - (b) If *state* = 2: append to *inorder*; push (*node*, 3); if right child exists push (right, 1).
 - (c) If *state* = 3: append to *postorder*.

Dry run

Dry Run. Tree: root 1; left 2; right 3.

- Pop (1, 1): pre= [1]; push (1, 2); push (2, 1).
- Pop (2, 1): pre= [1, 2]; push (2, 2) (no left).
- Pop (2, 2): in= [2]; push (2, 3) (no right).
- Pop (2, 3): post= [2].
- Pop (1, 2): in= [2, 1]; push (1, 3); push (3, 1).
- Pop (3, 1): pre= [1, 2, 3]; push (3, 2). Pop (3, 2): in= [2, 1, 3]; push (3, 3). Pop (3, 3): post= [2, 3].
- Pop (1, 3): post= [2, 3, 1].
- Pre = 1, 2, 3; In = 2, 1, 3; Post = 2, 3, 1.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 void allThree(TreeNode* root, vector<int>& pre,
12               vector<int>& in, vector<int>& post) {
13     if (root == nullptr) return;
14     stack<pair<TreeNode*, int>> st;

```

```

15     st.push({root, 1});
16
17     while (!st.empty()) {
18         auto [node, state] = st.top();
19         st.pop();
20         if (state == 1) {
21             pre.push_back(node->val);
22             st.push({node, 2});
23             if (node->left) st.push({node->left, 1});
24         } else if (state == 2) {
25             in.push_back(node->val);
26             st.push({node, 3});
27             if (node->right) st.push({node->right, 1});
28         } else {
29             post.push_back(node->val);
30         }
31     }
32 }
33
34 int main() {
35     TreeNode* root = new TreeNode(1);
36     root->left = new TreeNode(2);
37     root->right = new TreeNode(3);
38
39     vector<int> pre, in, post;
40     allThree(root, pre, in, post);
41
42     auto printv = [](const string& n, vector<int>& v) {
43         cout << n << ": ";
44         for (int x : v) cout << x << " ";
45         cout << "\n";
46     };
47     printv("pre ", pre);
48     printv("in  ", in);
49     printv("post", post);
50     return 0;
51 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is pushed and popped exactly three times, a constant factor. **Space Complexity.** $O(h)$ for the stack, plus $O(n)$ for the three output arrays.

Key Takeaway

The state 1/2/3 per stack frame is exactly what a recursive DFS stores implicitly as “which line am I on when I return.” Making that state explicit is a general technique for merging several traversal-order tasks into one pass.

1.9 Maximum Depth (Height) of a Binary Tree

Problem Statement. Given the root of a binary tree, return its maximum depth: the number of nodes along the longest path from the root down to a leaf.

Pattern Recognition

“Height,” “depth,” “longest path to a leaf” is the archetypal **bottom-up** recursion, and the first concrete instance of this chapter’s core rhythm. A node’s height depends on its children’s heights, so you must have both child answers *before* you can answer for the node – that ordering *is* postorder.

Understanding the problem carefully

The height of a tree rooted at a node is 1 (for the node itself) plus the height of its taller subtree. An empty tree has height 0. This is the purest example of “ask both children, combine”: the combine step is simply $1 + \max(\text{left height}, \text{right height})$.

Height = 1 + max of the Two Child Heights

Recurse to the left and right children, obtain their heights, and return one more than the larger. The base case (null returns 0) is what terminates the recursion and seeds the count. No global state is needed: the answer flows straight up the return values.

Algorithm

1. HEIGHT(*node*):
 - (a) If *node* is null, return 0.
 - (b) $L = \text{HEIGHT}(\text{node.left}); R = \text{HEIGHT}(\text{node.right})$.
 - (c) Return $1 + \max(L, R)$.

Dry run

Dry Run. Tree: root 1; left 2 (leaf); right 3 whose right child is 4 (leaf).

- Height(2)= 1 (both children null $\rightarrow 1 + \max(0, 0)$).
- Height(4)= 1. Height(3)= $1 + \max(0, 1) = 2$.
- Height(1)= $1 + \max(1, 2) = 3$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 int maxDepth(TreeNode* node) {
12     if (node == nullptr) return 0;
13     int L = maxDepth(node->left);
14     int R = maxDepth(node->right);
15     return 1 + max(L, R);
16 }
```

```

17
18 int main() {
19     TreeNode* root = new TreeNode(1);
20     root->left = new TreeNode(2);
21     root->right = new TreeNode(3);
22     root->right->right = new TreeNode(4);
23
24     cout << "Max depth: " << maxDepth(root) << "\n";
25     return 0;
26 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node contributes $O(1)$ combine work. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

Height is the “hello world” of bottom-up tree recursion, and the exact subroutine reused inside balanced-tree, diameter, and max-path-sum problems. Each of those is “compute height, but also check or update something extra along the way.”

1.10 Check for a Balanced Binary Tree

Problem Statement. A binary tree is height-balanced if, for *every* node, the heights of its left and right subtrees differ by at most 1. Given the root, determine whether the tree is balanced.

Pattern Recognition

The naive reading – “for every node, compute both subtree heights and compare” – is an $O(n^2)$ trap on a skewed tree, because height is recomputed from scratch at every node. The keyword “*for every node*” combined with “*heights*” says: compute height once, bottom-up, and **fold the balance check into that same pass.**

Understanding the problem carefully

Height and the balance condition are computed from the same recursion. If each recursive call returns the height of its subtree, the parent can, in $O(1)$, check whether its two child heights differ by more than 1 before returning its own height. A single sentinel value (say -1) propagated upward can signal “a violation was found below” so the whole recursion short-circuits.

Return Height, or -1 to Mean “Already Unbalanced”

Write a helper that returns the subtree height normally, but returns -1 the moment it detects imbalance – either because a child already returned -1 , or because the two child heights differ by more than 1. The final answer is “balanced” iff the root’s call returns something other than -1 . One pass, $O(n)$.

Algorithm

1. $CHECK(node)$:
 - (a) If $node$ is null, return 0.
 - (b) $L = CHECK(node.left)$; if $L = -1$ return -1 .

- (c) $R = \text{CHECK}(\text{node.right})$; if $R = -1$ return -1 .
- (d) If $|L - R| > 1$ return -1 ; else return $1 + \max(L, R)$.

2. The tree is balanced iff $\text{CHECK}(\text{root}) \neq -1$.

Dry run

Dry Run. Tree: root 1; left 2 with a left child 3 that has a left child 4; right subtree of 1 empty.

- $\text{Height}(4) = 1$, $\text{Height}(3) = 1 + \max(1, 0) = 2$.
- At 2: left height 2, right height 0; $|2 - 0| = 2 > 1 \Rightarrow$ return -1 .
- -1 propagates to 1; answer: **not balanced**.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 int check(TreeNode* node) {
12     if (node == nullptr) return 0;
13     int L = check(node->left);
14     if (L == -1) return -1;
15     int R = check(node->right);
16     if (R == -1) return -1;
17     if (abs(L - R) > 1) return -1;
18     return 1 + max(L, R);
19 }
20
21 bool isBalanced(TreeNode* root) {
22     return check(root) != -1;
23 }
24
25 int main() {
26     TreeNode* root = new TreeNode(1);
27     root->left = new TreeNode(2);
28     root->left->left = new TreeNode(3);
29     root->left->left->left = new TreeNode(4);
30
31     cout << (isBalanced(root) ? "balanced" : "not balanced") << "\n";
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. $O(n)$ – height and the balance test share one postorder pass, versus $O(n^2)$ if height were recomputed per node. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

Overloading a return value with a sentinel ($-1 = \text{"violation found"}$) is a clean way to combine “compute a quantity” with “check a property” in one traversal, avoiding the quadratic blow-up of recomputing the quantity at every node.

1.11 Diameter of a Binary Tree

Problem Statement. The diameter of a binary tree is the length (number of edges) of the longest path between any two nodes. The path need not pass through the root. Given the root, return the diameter.

Pattern Recognition

“Longest path between any two nodes, not necessarily through the root” is the classic “return one thing, track another” pattern. At each node the *best path passing through it* is left height + right height, but what the node must *return* to its parent is only its own height. Two different quantities: a **global maximum** for the answer, a **return value** for the parent.

Understanding the problem carefully

Any path has a single highest node – its “peak.” The path through a given peak descends into the left subtree and the right subtree, so its length is (height of left subtree) + (height of right subtree) edges. The overall diameter is the maximum of that quantity over all possible peaks. Since every node is the peak of exactly one candidate path, we evaluate this candidate at every node during a height computation.

Compute Height, Update a Global Diameter Along the Way

Run the ordinary height recursion. At each node, before returning, update a global answer with $L + R$ (the through-this-node path length in edges). Then return $1 + \max(L, R)$ to the parent as usual. The node contributes its own candidate to the global max but hands the parent only what the parent needs.

Algorithm

1. Keep a global $best = 0$.
2. $HEIGHT(node)$:
 - (a) If null, return 0.
 - (b) $L = HEIGHT(left)$, $R = HEIGHT(right)$.
 - (c) $best = \max(best, L + R)$.
 - (d) Return $1 + \max(L, R)$.
3. Answer is $best$.

Dry run

Dry Run. Tree: root 1; left 2 with children 4, 5; right 3.

- $Height(4) = Height(5) = 1$. At 2: $L = 1, R = 1$, $best = \max(0, 2) = 2$; return 2.

- Height(3)= 1. At 1: $L = 2, R = 1$, best= $\max(2, 3) = 3$; return 3.
- Diameter = 3 edges (path 4–2–1–3).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 int best;
12
13 int height(TreeNode* node) {
14     if (node == nullptr) return 0;
15     int L = height(node->left);
16     int R = height(node->right);
17     best = max(best, L + R);           // path through this node, in edges
18     return 1 + max(L, R);
19 }
20
21 int diameterOfBinaryTree(TreeNode* root) {
22     best = 0;
23     height(root);
24     return best;
25 }
26
27 int main() {
28     TreeNode* root = new TreeNode(1);
29     root->left = new TreeNode(2);
30     root->right = new TreeNode(3);
31     root->left->left = new TreeNode(4);
32     root->left->right = new TreeNode(5);
33
34     cout << "Diameter: " << diameterOfBinaryTree(root) << "\n";
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one height pass with $O(1)$ extra work per node. A naive “compute height twice at every node” solution would be $O(n^2)$. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

“Return one value to the parent, update a separate global for the answer” is the reusable shape for any tree problem where the optimum can bend through a node rather than continue straight down. The very next section, maximum path sum, is the same idea with values instead of heights.

1.12 Maximum Path Sum

Problem Statement. A path is any sequence of nodes connected by edges, where each adjacent pair shares an edge and no node repeats; it need not pass through the root. The path sum is the total of its node values. Given the root (values may be negative), return the maximum path sum.

Pattern Recognition

This is the diameter problem with *values instead of edge counts*, plus one subtlety: because values can be negative, a subtree that would *lower* the total should be dropped entirely. The keywords “*maximum*,” “*path*,” “*may be negative*” say: bottom-up, track a global best through each node, but clamp negative child contributions to zero.

Understanding the problem carefully

At any node, the best path with that node as its peak is: the node’s value, plus the best downward path into the left subtree (if positive), plus the best downward path into the right subtree (if positive). That is the candidate we compare against a global best. But the value we *return* to the parent can only continue *one* way – a path cannot fork at the parent – so we return the node’s value plus the better of its two downward gains.

Return a Straight-Down Gain, Score a Bent Path

Let $\text{GAIN}(\text{node})$ return the maximum sum of a path that starts at *node* and goes strictly downward. Compute $l = \max(0, \text{GAIN}(\text{left}))$ and $r = \max(0, \text{GAIN}(\text{right}))$ (the $\max(0, \cdot)$ drops harmful subtrees). Update the global best with $\text{node.val} + l + r$ (the path bending through this node). Return $\text{node.val} + \max(l, r)$ to the parent (only one branch may continue).

Algorithm

1. Global *best* = $-\infty$.
2. $\text{GAIN}(\text{node})$:
 - (a) If null, return 0.
 - (b) $l = \max(0, \text{GAIN}(\text{left}))$; $r = \max(0, \text{GAIN}(\text{right}))$.
 - (c) $\text{best} = \max(\text{best}, \text{node.val} + l + r)$.
 - (d) Return $\text{node.val} + \max(l, r)$.
3. Answer is *best*.

Dry run

Dry Run. Tree: root -10 ; left 9; right 20 with children 15, 7.

- $\text{Gain}(9) = 9$, $\text{Gain}(15) = 15$, $\text{Gain}(7) = 7$.
- At 20: $l = 15$, $r = 7$; $\text{best} = \max(-\infty, 20 + 15 + 7) = 42$; return $20 + 15 = 35$.
- At -10 : $l = \max(0, 9) = 9$, $r = \max(0, 35) = 35$; $\text{best} = \max(42, -10 + 9 + 35) = 42$; return $-10 + 35 = 25$.
- Maximum path sum = 42 (path 15–20–7).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 int best;
12
13 int gain(TreeNode* node) {
14     if (node == nullptr) return 0;
15     int l = max(0, gain(node->left));    // drop harmful subtrees
16     int r = max(0, gain(node->right));
17     best = max(best, node->val + l + r); // path bending through node
18     return node->val + max(l, r);       // parent can continue one way
19 }
20
21 int maxPathSum(TreeNode* root) {
22     best = INT_MIN;
23     gain(root);
24     return best;
25 }
26
27 int main() {
28     TreeNode* root = new TreeNode(-10);
29     root->left = new TreeNode(9);
30     root->right = new TreeNode(20);
31     root->right->left = new TreeNode(15);
32     root->right->right = new TreeNode(7);
33
34     cout << "Max path sum: " << maxPathSum(root) << "\n";
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one postorder pass. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

The $\max(0, \cdot)$ clamp – “take a subtree’s contribution only if it helps” – is the one new idea beyond the diameter template. Whenever a downward contribution can be negative, clamping it to zero is how greedy “skip the bad branch” decisions are expressed inside a bottom-up recursion.

1.13 Check if Two Trees Are Identical

Problem Statement. Given the roots of two binary trees, determine whether they are structurally identical and have the same node values at every corresponding position.

Pattern Recognition

“Are these two trees the same?” is a **simultaneous traversal**: walk both trees in lockstep and compare node against node. The recursion is the same “ask both children” shape, but applied to a pair of nodes at once, with equality being the conjunction of “roots match” and “left subtrees match” and “right subtrees match.”

Understanding the problem carefully

Two trees are identical exactly when their roots hold equal values and, recursively, their left subtrees are identical and their right subtrees are identical. The base cases handle the ends of branches: two nulls are identical; one null and one node are not.

Lockstep Recursion with a Three-Way Base Case

Compare node p and node q together:

- If both are null: identical here (return true).
- If exactly one is null, or their values differ: not identical (return false).
- Otherwise: return “left pair identical” **and** “right pair identical.”

Algorithm

1. SAME(p, q):
 - (a) If p and q are both null, return true.
 - (b) If exactly one is null, return false.
 - (c) If $p.val \neq q.val$, return false.
 - (d) Return SAME($p.left, q.left$) **and** SAME($p.right, q.right$).

Dry run

Dry Run. Tree A: root 1, left 2, right 3. Tree B: root 1, left 2, right 4.

- Roots 1 = 1: recurse on left pair and right pair.
- Left pair 2, 2: equal, children all null $\Rightarrow$ true.
- Right pair 3, 4: $3 \neq 4 \Rightarrow$ false.
- Conjunction: true **and** false = **false**. Not identical.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };

```

```

10
11 bool isSameTree(TreeNode* p, TreeNode* q) {
12     if (p == nullptr && q == nullptr) return true;
13     if (p == nullptr || q == nullptr) return false;
14     if (p->val != q->val) return false;
15     return isSameTree(p->left, q->left) &&
16           isSameTree(p->right, q->right);
17 }
18
19 int main() {
20     TreeNode* a = new TreeNode(1);
21     a->left = new TreeNode(2);
22     a->right = new TreeNode(3);
23
24     TreeNode* b = new TreeNode(1);
25     b->left = new TreeNode(2);
26     b->right = new TreeNode(4);
27
28     cout << (isSameTree(a, b) ? "identical" : "different") << "\n";
29     return 0;
30 }

```

Complexity Analysis

Time Complexity. $O(n)$ where n is the size of the smaller tree – the recursion stops at the first mismatch or null. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

Comparing two structures by recursing on both in lockstep, with a careful null/null vs. null/non-null base case, is a template that also underlies “symmetric tree” (compare a tree against its own mirror) and subtree-of checks.

1.14 Zigzag (Spiral) Level Order Traversal

Problem Statement. Given the root of a binary tree, return its zigzag level order traversal: the first level left-to-right, the second right-to-left, the third left-to-right, and so on, alternating direction each level.

Pattern Recognition

“Zigzag,” “spiral,” “alternate direction each level” is plain **level-order BFS** with one extra bit of state: a flag that flips every level and controls whether the current level is emitted forwards or backwards. The traversal structure does not change at all.

Understanding the problem carefully

BFS already produces nodes grouped by level, left to right. To zigzag, we only need to reverse the ordering of *every other* level before adding it to the answer. The cleanest way is to keep the standard left-to-right BFS but, when the direction flag says “right-to-left,” place each node’s value into its position from the back of the current level’s list.

Standard BFS, Reverse Alternate Levels

Do an ordinary level-order traversal. Maintain a boolean *leftToRight*. For each level of size s , allocate a list of size s ; when *leftToRight* is true, write the i -th dequeued node at index i ; when false, write it at index $s - 1 - i$. Flip *leftToRight* after each level.

Algorithm

1. If root is null, return empty. Push root; *leftToRight* = true.
2. While the queue is non-empty:
 - (a) s = queue size; make a level array of length s .
 - (b) For $i = 0 \dots s - 1$: pop node; index = *leftToRight* ? i : $s - 1 - i$; store value at that index; push children.
 - (c) Append the level array; flip *leftToRight*.

Dry run

Dry Run. Tree: root 1; children 2, 3; 2's children 4, 5; 3's children 6, 7.

- Level 0 (L→R): [1].
- Level 1 (R→L): nodes dequeued 2, 3 written at indices 1, 0 $\Rightarrow$ [3, 2].
- Level 2 (L→R): [4, 5, 6, 7].
- Result: [[1], [3, 2], [4, 5, 6, 7]].

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
12     vector<vector<int>> ans;
13     if (root == nullptr) return ans;
14
15     queue<TreeNode*> q;
16     q.push(root);
17     bool leftToRight = true;
18     while (!q.empty()) {
19         int s = q.size();
20         vector<int> level(s);
21         for (int i = 0; i < s; i++) {
22             TreeNode* node = q.front();
23             q.pop();
24             int idx = leftToRight ? i : (s - 1 - i);
25             level[idx] = node->val;

```

```

26         if (node->left) q.push(node->left);
27         if (node->right) q.push(node->right);
28     }
29     leftToRight = !leftToRight;
30     ans.push_back(level);
31 }
32 return ans;
33 }
34
35 int main() {
36     TreeNode* root = new TreeNode(1);
37     root->left = new TreeNode(2);
38     root->right = new TreeNode(3);
39     root->left->left = new TreeNode(4);
40     root->left->right = new TreeNode(5);
41     root->right->left = new TreeNode(6);
42     root->right->right = new TreeNode(7);
43
44     for (auto& lvl : zigzagLevelOrder(root)) {
45         for (int x : lvl) cout << x << " ";
46         cout << "\n";
47     }
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one BFS pass; the index arithmetic is $O(1)$ per node. **Space Complexity.** $O(n)$ for the queue and output.

Key Takeaway

Many “fancy-looking” traversals are ordinary BFS with a single extra piece of state controlling how each level is read off. Spot the underlying level-order engine and the decoration – here, a direction flag – becomes trivial.

1.15 Boundary Traversal of a Binary Tree

Problem Statement. Given the root of a binary tree, return an anti-clockwise boundary traversal: the left boundary (top-down, excluding leaves), then all leaves left-to-right, then the right boundary (bottom-up, excluding leaves). The root is included once.

Pattern Recognition

“Boundary,” “outline,” “perimeter” decomposes into three independent walks that must be stitched without double-counting: the left edge going down, the leaves going across, and the right edge coming back up. The only real care is avoiding overlap where these parts meet – which is why leaves are deliberately excluded from the two edges.

Understanding the problem carefully

The perimeter of the tree consists of three disjoint pieces. The **left boundary** is the path you get by always going left (or right if there is no left child), from the root’s left child down, stopping before leaves. The **leaves** are every node with no children, in left-to-right order. The

right boundary is the mirror – always go right (or left otherwise) – collected then reversed so it reads bottom-up. Excluding leaves from the two boundaries prevents a corner leaf from being listed twice.

Three Walks, No Overlap

(1) Add the root (unless it is itself a leaf). (2) Walk the left boundary top-down, adding non-leaf nodes. (3) Add all leaves via any DFS, left-to-right. (4) Walk the right boundary top-down into a temporary list, adding non-leaf nodes, then append that list *reversed*. Leaves are skipped in steps 2 and 4 precisely so step 3 owns them exclusively.

Algorithm

1. If root is not a leaf, add root's value.
2. Left boundary: start at root.left; while the node exists, if it is not a leaf add it; move to its left child, or right child if no left.
3. Leaves: DFS the whole tree; whenever a node has no children, add it.
4. Right boundary: start at root.right; collect non-leaf nodes top-down (preferring right children) into a list; append that list reversed.

Dry run

Dry Run. Tree: root 1; left 2 (children 4,5); right 3 (right child 6, whose left child 7).

- Root 1 (not a leaf): add 1.
- Left boundary from 2: 2 not a leaf, add 2; go to 4 – a leaf, stop. Boundary so far: 1, 2.
- Leaves L→R: 4, 5, 7. Now: 1, 2, 4, 5, 7.
- Right boundary from 3: 3 not leaf add 3; go to 6, not leaf add 6; 7 is a leaf, stop. List [3, 6] reversed = [6, 3].
- Result: 1, 2, 4, 5, 7, 6, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 bool isLeaf(TreeNode* n) {
12     return n->left == nullptr && n->right == nullptr;
13 }
14
15 void addLeaves(TreeNode* n, vector<int>& out) {
16     if (n == nullptr) return;
17     if (isLeaf(n)) { out.push_back(n->val); return; }

```

```

18     addLeaves(n->left, out);
19     addLeaves(n->right, out);
20 }
21
22 vector<int> boundaryTraversal(TreeNode* root) {
23     vector<int> out;
24     if (root == nullptr) return out;
25     if (!isLeaf(root)) out.push_back(root->val);
26
27     // left boundary (top-down, non-leaf)
28     TreeNode* cur = root->left;
29     while (cur) {
30         if (!isLeaf(cur)) out.push_back(cur->val);
31         cur = cur->left ? cur->left : cur->right;
32     }
33
34     // all leaves, left to right
35     addLeaves(root, out);
36
37     // right boundary (top-down into temp, then reversed)
38     vector<int> tmp;
39     cur = root->right;
40     while (cur) {
41         if (!isLeaf(cur)) tmp.push_back(cur->val);
42         cur = cur->right ? cur->right : cur->left;
43     }
44     for (int i = (int)tmp.size() - 1; i >= 0; i--) out.push_back(tmp[i]);
45     return out;
46 }
47
48 int main() {
49     TreeNode* root = new TreeNode(1);
50     root->left = new TreeNode(2);
51     root->left->left = new TreeNode(4);
52     root->left->right = new TreeNode(5);
53     root->right = new TreeNode(3);
54     root->right->right = new TreeNode(6);
55     root->right->right->left = new TreeNode(7);
56
57     for (int x : boundaryTraversal(root)) cout << x << " ";
58     cout << "\n";
59     return 0;
60 }

```

Complexity Analysis

Time Complexity. $O(n)$ – the leaf DFS is $O(n)$ and each boundary walk is $O(h)$. **Space Complexity.** $O(n)$ for the output and $O(h)$ recursion for the leaf pass.

Key Takeaway

Boundary problems are less about clever algorithms and more about a clean decomposition with disciplined de-duplication at the seams. Deciding up front that “leaves belong to exactly one of the three parts” is what keeps the corners from being counted twice.

1.16 Vertical Order Traversal

Problem Statement. Assign the root a horizontal distance (HD) of 0; a left child has HD one less than its parent, a right child one more. Return nodes grouped by HD from leftmost to rightmost. Within the same HD, order by depth (top to bottom); nodes at the same HD *and* same depth are ordered by value.

Pattern Recognition

“Vertical,” “columns,” “horizontal distance” says: tag each node with a **(HD, depth)** coordinate, collect everything into a map keyed by HD, and sort within each column. The traversal is ordinary BFS or DFS; the work is in the coordinate bookkeeping and the tie-breaking rules.

Understanding the problem carefully

Imagine dropping vertical lines through the tree. Every node falls on one line, identified by its horizontal distance. Reading the answer means listing lines left to right, and within a line, top to bottom. The precise LeetCode-style rule adds a final tie-break: if two nodes share both HD and depth, the smaller value comes first. Storing each node as a triple (HD, depth, value) and sorting makes all three rules fall out of one comparison.

Order by (HD, then depth, then value)

Traverse the tree recording, for each node, the triple $(HD, depth, val)$: root is $(0, 0, val)$, a left child inherits $(HD - 1, depth + 1)$, a right child $(HD + 1, depth + 1)$. Group into a map from HD to the list of $(depth, val)$ pairs; sort each list by $(depth, val)$; emit columns in increasing HD.

Algorithm

1. BFS/DFS from the root carrying $(HD, depth)$; append $(depth, val)$ into $cols[HD]$.
2. For each HD in increasing order, sort its list by $(depth, val)$ and output the values.

Dry run

Dry Run. Tree: root 1; left 2; right 3; 2’s right child 4; 3’s left child 5 (so 4 and 5 both land on HD 0).

- $1 : (0, 0)$; $2 : (-1, 1)$; $3 : (1, 1)$; $4 : (0, 2)$; $5 : (0, 2)$.
- HD -1 : $[2]$. HD 0 : $\{(0, 1), (2, 4), (2, 5)\}$ sorted by $(depth, val) \rightarrow 1, 4, 5$. HD 1 : $[3]$.
- Result: $[[2], [1, 4, 5], [3]]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;

```

```

8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 void dfs(TreeNode* node, int hd, int depth,
12         map<int, vector<pair<int,int>>>& cols) {
13     if (node == nullptr) return;
14     cols[hd].push_back({depth, node->val});
15     dfs(node->left,  hd - 1, depth + 1, cols);
16     dfs(node->right, hd + 1, depth + 1, cols);
17 }
18
19 vector<vector<int>> verticalTraversal(TreeNode* root) {
20     map<int, vector<pair<int,int>>> cols;    // hd -> (depth, val)
21     dfs(root, 0, 0, cols);
22
23     vector<vector<int>> ans;
24     for (auto& [hd, vec] : cols) {          // map iterates hd
25         ascending                          // by depth, then value
26         sort(vec.begin(), vec.end());
27         vector<int> col;
28         for (auto& [d, v] : vec) col.push_back(v);
29         ans.push_back(col);
30     }
31     return ans;
32 }
33
34 int main() {
35     TreeNode* root = new TreeNode(1);
36     root->left = new TreeNode(2);
37     root->right = new TreeNode(3);
38     root->left->right = new TreeNode(4);
39     root->right->left = new TreeNode(5);
40
41     for (auto& col : verticalTraversal(root)) {
42         for (int x : col) cout << x << " ";
43         cout << "\n";
44     }
45     return 0;
46 }

```

Complexity Analysis

Time Complexity. $O(n \log n)$ – dominated by sorting the columns (total n entries across all columns). **Space Complexity.** $O(n)$ for the map and the recursion stack.

Key Takeaway

Tagging nodes with coordinates and offloading the ordering to a sorted map plus a per-column sort turns a geometric-sounding problem into bookkeeping. Top view and bottom view (next sections) reuse the same HD tag with a simpler rule for which node on each line “wins.”

1.17 Top View of a Binary Tree

Problem Statement. The top view is the set of nodes visible when the tree is viewed from directly above: for each horizontal distance (HD), the single *topmost* node on that vertical line.

Return these nodes ordered left to right by HD.

Pattern Recognition

Reusing the HD tag from vertical order, “*top view / viewed from above*” means: on each vertical line, keep only the node encountered *first when scanning top-down*. A **BFS** scans strictly in increasing depth, so the first node BFS places on a given HD is automatically the topmost – record an HD only the first time it appears.

Understanding the problem carefully

Looking down from above, a node is hidden if some other node shares its horizontal distance but sits higher. Because BFS processes level 0 before level 1 before level 2, the earliest time an HD is seen during BFS corresponds to the highest node on that line. So we simply record the first value per HD and ignore all later ones.

BFS, First Node per HD Wins

Run level-order BFS carrying each node’s HD (root 0, left -1 , right $+1$). Keep a map from HD to value, writing an entry *only if that HD is not already present*. BFS’s increasing-depth order guarantees the first write for each HD is the topmost node. Emit the map in ascending HD order.

Why BFS, not DFS: a DFS might reach a deep node on some HD before a shallower node on the same HD (down a different branch), wrongly recording the deeper one. BFS’s level-by-level order removes that hazard.

Algorithm

1. BFS from root with a queue of $(node, HD)$; root’s $HD = 0$.
2. For each dequeued $(node, HD)$: if HD not yet in the map, set $map[HD] = node.val$. Enqueue $(left, HD - 1)$ and $(right, HD + 1)$.
3. Output the map’s values in ascending HD.

Dry run

Dry Run. Tree: root 1; left 2 (right child 4); right 3.

- $(1, 0)$: $map\{0:1\}$; enqueue $(2, -1), (3, 1)$.
- $(2, -1)$: $map\{-1:2, 0:1\}$; enqueue $(4, 0)$.
- $(3, 1)$: map adds $1:3$.
- $(4, 0)$: HD 0 already present – skip.
- Ascending HD: 2, 1, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;

```

```

6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 vector<int> topView(TreeNode* root) {
12     vector<int> ans;
13     if (root == nullptr) return ans;
14
15     map<int, int> topAtHD;           // hd -> first (topmost) value
16     queue<pair<TreeNode*, int>> q;
17     q.push({root, 0});
18     while (!q.empty()) {
19         auto [node, hd] = q.front();
20         q.pop();
21         if (topAtHD.find(hd) == topAtHD.end())
22             topAtHD[hd] = node->val;    // first sighting wins
23         if (node->left) q.push({node->left, hd - 1});
24         if (node->right) q.push({node->right, hd + 1});
25     }
26     for (auto& [hd, v] : topAtHD) ans.push_back(v);
27     return ans;
28 }
29
30 int main() {
31     TreeNode* root = new TreeNode(1);
32     root->left = new TreeNode(2);
33     root->right = new TreeNode(3);
34     root->left->right = new TreeNode(4);
35
36     for (int x : topView(root)) cout << x << " ";
37     cout << "\n";
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(n \log n)$ – n BFS steps, each doing an $O(\log n)$ map operation (or $O(n)$ overall with a hash map plus a final sort). **Space Complexity.** $O(n)$ for the queue and map.

Key Takeaway

“First per HD under BFS” is the whole trick for top view. The insistence on BFS over DFS is the subtle correctness point: only a strictly increasing-depth scan makes “first seen” equal to “topmost.”

1.18 Bottom View of a Binary Tree

Problem Statement. The bottom view is the set of nodes visible when the tree is viewed from directly below: for each horizontal distance (HD), the *lowest* node on that vertical line (ties broken by the later node in level order). Return these ordered left to right by HD.

Pattern Recognition

Bottom view is top view with one word changed: keep the *last* node per HD instead of the first. Under BFS – which scans top-down – the last node written to a given HD is the deepest, exactly the one visible from below. So the code is identical except we **always overwrite** the map entry rather than writing only once.

Understanding the problem carefully

From below, on each vertical line the visible node is the lowest one. Because BFS visits nodes in non-decreasing depth, repeatedly overwriting the value stored for an HD leaves, at the end, the value of the last (hence deepest) node BFS placed there. The left-to-right order within a level also gives the standard tie-break for two nodes at the same HD and depth.

BFS, Last Node per HD Wins

Run the same level-order BFS with HD tags as top view, but write $map[HD] = node.val$ *unconditionally* every time. Later (deeper) nodes overwrite earlier ones, so each HD ends holding its bottommost node. Emit ascending by HD.

Algorithm

1. BFS from root with $(node, HD)$; root's $HD = 0$.
2. For each dequeued $(node, HD)$: set $map[HD] = node.val$ (overwrite). Enqueue $(left, HD - 1)$, $(right, HD + 1)$.
3. Output the map's values in ascending HD.

Dry run

Dry Run. Tree: root 1; left 2 (right child 4); right 3.

- $(1, 0)$: $map\{0:1\}$; enqueue $(2, -1), (3, 1)$.
- $(2, -1)$: $map\{-1:2\}$; enqueue $(4, 0)$.
- $(3, 1)$: map adds $1:3$.
- $(4, 0)$: overwrite HD 0 with 4.
- Ascending HD: 2, 4, 3 (note HD 0 now shows 4, not 1).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 vector<int> bottomView(TreeNode* root) {
```

```

12     vector<int> ans;
13     if (root == nullptr) return ans;
14
15     map<int, int> lowAtHD;           // hd -> last (lowest) value
16     queue<pair<TreeNode*, int>> q;
17     q.push({root, 0});
18     while (!q.empty()) {
19         auto [node, hd] = q.front();
20         q.pop();
21         lowAtHD[hd] = node->val;    // always overwrite
22         if (node->left) q.push({node->left, hd - 1});
23         if (node->right) q.push({node->right, hd + 1});
24     }
25     for (auto& [hd, v] : lowAtHD) ans.push_back(v);
26     return ans;
27 }
28
29 int main() {
30     TreeNode* root = new TreeNode(1);
31     root->left = new TreeNode(2);
32     root->right = new TreeNode(3);
33     root->left->right = new TreeNode(4);
34
35     for (int x : bottomView(root)) cout << x << " ";
36     cout << "\n";
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(n \log n)$ with an ordered map (or $O(n)$ with a hash map and a final sort). **Space Complexity.** $O(n)$ for the queue and map.

Key Takeaway

Top view and bottom view differ by exactly one decision – keep the first versus the last node per HD. Recognising that a whole family of “view” problems shares one BFS-with-HD engine, differing only in the per-line selection rule, is the real lesson.

1.19 Left View and Right View of a Binary Tree

Problem Statement. The right view is the sequence of nodes visible from the right side of the tree: the *last* node at each level (top to bottom). The left view is the *first* node at each level. Return both.

Pattern Recognition

“View from the side,” “first/last node per level” is level-order selection: for each level, the rightmost node is the right view, the leftmost is the left view. This is even simpler than the HD-based views – there is no horizontal distance, only “which node ends (or begins) each level.”

Understanding the problem carefully

Standing to the right of the tree, on each level you see only the node furthest right, because it blocks the ones behind it. So the right view is just “the last node of each level.” Equivalently,

with a depth-first approach that visits *right child before left*, the first node seen at each new depth is the right-view node – which gives an elegant $O(h)$ recursion.

First Node Seen per Depth, Right-First DFS

Recurse carrying the current depth. Visit the *right* child before the left. The first time the recursion reaches a given depth, the node it is at is the rightmost at that depth – record it. For the left view, do the same but visit the *left* child first.

Algorithm (right view)

1. DFS(*node*, *depth*, *out*):
 - (a) If *node* is null, return.
 - (b) If *depth* = size of *out*: this is the first node reached at this depth – append its value.
 - (c) DFS(*node*.right, *depth*+1); DFS(*node*.left, *depth*+1).

Dry run

Dry Run. Tree: root 1; left 2 (right child 5); right 3.

- DFS(1,0): *out* empty, *depth* 0 = 0 $\Rightarrow$ record 1. Recurse right into 3, then left into 2.
- DFS(3,1): *depth* 1 = size 1 $\Rightarrow$ record 3. (No children.)
- DFS(2,1): *depth* 1 $\neq$ size 2 – skip. Recurse right into 5.
- DFS(5,2): *depth* 2 = size 2 $\Rightarrow$ record 5.
- Right view: 1,3,5. (Left view by symmetry: 1,2,5.)

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 void rightDFS(TreeNode* node, int depth, vector<int>& out) {
12     if (node == nullptr) return;
13     if (depth == (int)out.size()) out.push_back(node->val);
14     rightDFS(node->right, depth + 1, out);    // right first
15     rightDFS(node->left, depth + 1, out);
16 }
17
18 void leftDFS(TreeNode* node, int depth, vector<int>& out) {
19     if (node == nullptr) return;
20     if (depth == (int)out.size()) out.push_back(node->val);
21     leftDFS(node->left, depth + 1, out);    // left first
22     leftDFS(node->right, depth + 1, out);
23 }

```

```

24
25 int main() {
26     TreeNode* root = new TreeNode(1);
27     root->left = new TreeNode(2);
28     root->right = new TreeNode(3);
29     root->left->right = new TreeNode(5);
30
31     vector<int> rv, lv;
32     rightDFS(root, 0, rv);
33     leftDFS(root, 0, lv);
34
35     cout << "Right view: "; for (int x : rv) cout << x << " "; cout <<
        "\n";
36     cout << "Left view: "; for (int x : lv) cout << x << " "; cout <<
        "\n";
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(n)$ – every node is visited once. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

The “record only when depth equals the current answer size” test is a compact way to detect “first node reached at a new depth” without an explicit level loop. Swapping the child-visit order flips left view to right view – another example of one algorithm, two mirrored variants.

1.20 Symmetric Binary Tree

Problem Statement. A binary tree is symmetric if it is a mirror image of itself around its centre: the left subtree is the mirror reflection of the right subtree. Given the root, determine whether the tree is symmetric.

Pattern Recognition

“Mirror,” “symmetric,” “reflection” is the identical-trees lockstep recursion (Section 1.13) with a twist: instead of comparing left-with-left and right-with-right, we compare the *outer* pair with each other and the *inner* pair with each other. Mirror matching crosses the two children.

Understanding the problem carefully

Symmetry is a statement about the left subtree versus the right subtree. Two subtrees are mirror images when their roots are equal *and* the left one’s left child mirrors the right one’s right child (the outer edges) *and* the left one’s right child mirrors the right one’s left child (the inner edges). The crossing – left.left with right.right, left.right with right.left – is what encodes “reflection” rather than “equality.”

Mirror-Compare: Outer with Outer, Inner with Inner

Define $\text{MIRROR}(a, b)$: they mirror iff both are null, or both are non-null with equal values *and* $\text{MIRROR}(a.\text{left}, b.\text{right})$ *and* $\text{MIRROR}(a.\text{right}, b.\text{left})$. The whole tree is symmetric iff $\text{MIRROR}(\text{root.left}, \text{root.right})$.

Algorithm

1. If root is null, return true.
2. MIRROR(a, b):
 - (a) If both null, return true; if exactly one null, return false.
 - (b) If $a.val \neq b.val$, return false.
 - (c) Return MIRROR($a.left, b.right$) **and** MIRROR($a.right, b.left$).
3. Return MIRROR($root.left, root.right$).

Dry run

Dry Run. Tree: root 1; left 2 (children 3,4); right 2 (children 4,3).

- Mirror($2_L, 2_R$): values equal. Compare outer ($2_L.left=3, 2_R.right=3$) and inner ($2_L.right=4, 2_R.left=4$).
- Outer 3,3: equal, children null $\rightarrow$ true. Inner 4,4: true.
- All true $\Rightarrow$ **symmetric**.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 bool mirror(TreeNode* a, TreeNode* b) {
12     if (a == nullptr && b == nullptr) return true;
13     if (a == nullptr || b == nullptr) return false;
14     if (a->val != b->val) return false;
15     return mirror(a->left, b->right) &&           // outer pair
16            mirror(a->right, b->left);           // inner pair
17 }
18
19 bool isSymmetric(TreeNode* root) {
20     if (root == nullptr) return true;
21     return mirror(root->left, root->right);
22 }
23
24 int main() {
25     TreeNode* root = new TreeNode(1);
26     root->left = new TreeNode(2);
27     root->right = new TreeNode(2);
28     root->left->left = new TreeNode(3);
29     root->left->right = new TreeNode(4);
30     root->right->left = new TreeNode(4);
31     root->right->right = new TreeNode(3);
32

```

```

33     cout << (isSymmetric(root) ? "symmetric" : "not symmetric") << "\n"
      ;
34     return 0;
35 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each pair of mirrored nodes is compared once. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

Symmetry is equality with the two children crossed. Recognising it as “the same-tree recursion, but mirror the child pairing” means you write it correctly the first time instead of tangling the four child comparisons.

1.21 Root-to-Node Path

Problem Statement. Given the root of a binary tree and a target node value (assume values are unique), return the sequence of node values on the path from the root down to the target.

Pattern Recognition

“Path from the root to a node,” “ancestors of a node” is a backtracking DFS: descend adding the current node to a running path; if this node is the target, stop and keep the path; if neither subtree contains the target, **remove the current node** before returning. The add-then-remove-on-failure rhythm is the essence of backtracking.

Understanding the problem carefully

There is exactly one path from the root to any node in a tree. We build it incrementally: push the current node onto the path, then check “is this the target, or is the target somewhere below on the left or right?” If yes, this node is genuinely on the path and stays. If the target is not here and not in either subtree, this node was a wrong turn and must be popped so it does not pollute the path for other branches.

Add on Entry, Pop on a Dead End

FIND(node, target, path): if node is null return false; push node.val onto path; if node.val equals target return true; if FIND(left) or FIND(right) returns true, return true (node stays on the path); otherwise pop node.val (dead end) and return false.

Algorithm

1. FIND(*node*, *target*, *path*):
 - (a) If *node* is null, return false.
 - (b) Push *node.val* onto *path*.
 - (c) If *node.val* = *target*, return true.
 - (d) If FIND(left) **or** FIND(right), return true.
 - (e) Pop *node.val*; return false.

Dry run

Dry Run. Tree: root 1; left 2 (children 4,5); right 3. Target 5.

- Push 1. Not 5. Recurse left into 2.
- Push 2. Not 5. Recurse left into 4: push 4, not 5, no children, pop 4, return false. Recurse right into 5: push 5, equals target, return true.
- 2 keeps (5 found below); 1 keeps.
- Path: 1, 2, 5.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 bool findPath(TreeNode* node, int target, vector<int>& path) {
12     if (node == nullptr) return false;
13     path.push_back(node->val);
14     if (node->val == target) return true;
15     if (findPath(node->left, target, path) ||
16         findPath(node->right, target, path))
17         return true;
18     path.pop_back();           // dead end: backtrack
19     return false;
20 }
21
22 int main() {
23     TreeNode* root = new TreeNode(1);
24     root->left = new TreeNode(2);
25     root->right = new TreeNode(3);
26     root->left->left = new TreeNode(4);
27     root->left->right = new TreeNode(5);
28
29     vector<int> path;
30     findPath(root, 5, path);
31     for (int x : path) cout << x << " ";
32     cout << "\n";
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. $O(n)$ – in the worst case every node is visited once. **Space Complexity.** $O(h)$ for the recursion stack and the path.

Key Takeaway

The push-on-entry / pop-on-failure pattern is backtracking in its simplest tree form, and it is the conceptual basis for the next section's lowest common ancestor and for any "list the ancestors" query.

1.22 Lowest Common Ancestor in a Binary Tree

Problem Statement. Given the root of a binary tree and two nodes p and q (both guaranteed present), return their lowest common ancestor: the deepest node that has both p and q as descendants (a node is a descendant of itself).

Pattern Recognition

"Lowest common ancestor" in a **general** binary tree (no BST ordering to exploit) is a single bottom-up recursion that reports back up which of p and q each subtree contains. The LCA is the unique node where the two targets are found on *different sides* – or which is itself one target with the other below it.

Understanding the problem carefully

Walk the tree once. Each recursive call answers: "does my subtree contain p or q (or one of them being me)?" by returning a non-null pointer if so. If a node finds one target in its left subtree and the other in its right subtree, that node is the meeting point – the LCA. If both are found on the same side, the LCA lies deeper on that side and is propagated up unchanged.

The Split Point Is the Answer

LCA(node): if node is null or equals p or q , return node. Recurse left and right. If *both* calls return non-null, the current node is where the two targets split – return it. Otherwise return whichever side was non-null (or null). The topmost node receiving non-null from both sides is the LCA.

Algorithm

1. LCA($node$):
 - (a) If $node$ is null or equals p or q , return $node$.
 - (b) $L = \text{LCA}(\text{left})$; $R = \text{LCA}(\text{right})$.
 - (c) If L and R are both non-null, return $node$.
 - (d) Return L if non-null, else R .

Dry run

Dry Run. Tree: root 3; left 5 (children 6, 2); right 1. Find LCA of 6 and 2.

- At 5: left call at 6 returns 6; right call at 2 returns 2. Both non-null $\Rightarrow$ 5 is the LCA; return 5.
- At 3: left returns 5, right (subtree 1) returns null. Only left non-null $\Rightarrow$ return 5.
- LCA = 5.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 TreeNode* lca(TreeNode* node, TreeNode* p, TreeNode* q) {
12     if (node == nullptr || node == p || node == q) return node;
13     TreeNode* L = lca(node->left, p, q);
14     TreeNode* R = lca(node->right, p, q);
15     if (L != nullptr && R != nullptr) return node; // split point
16     return L != nullptr ? L : R;
17 }
18
19 int main() {
20     TreeNode* root = new TreeNode(3);
21     root->left = new TreeNode(5);
22     root->right = new TreeNode(1);
23     TreeNode* p = root->left->left = new TreeNode(6);
24     TreeNode* q = root->left->right = new TreeNode(2);
25
26     cout << "LCA: " << lca(root, p, q)->val << "\n";
27     return 0;
28 }

```

Complexity Analysis

Time Complexity. $O(n)$ – a single traversal. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

Returning a “found” pointer up the recursion and declaring the LCA at the node where left and right both report success is a one-pass gem. Unlike the BST version, no value ordering is used – only the tree’s shape – which is why it works on any binary tree.

1.23 Maximum Width of a Binary Tree

Problem Statement. The width of a level is the number of positions between its leftmost and rightmost non-null nodes, *including* the null gaps between them (as if the tree were a complete binary tree). Return the maximum width over all levels.

Pattern Recognition

The phrase “*including null gaps*” is the whole difficulty: counting only present nodes is wrong. The fix is to give every node a **positional index** as if the tree were complete – a node at index i has children at $2i$ and $2i + 1$ – and measure each level’s width as (rightmost index – leftmost index + 1). This is BFS with indices.

Understanding the problem carefully

In a complete binary tree numbered from the root as index 0, a node at index i places its left child at $2i + 1$ and right child at $2i + 2$. These indices encode horizontal position *including* the empty slots. So for each level, the first and last real nodes carry indices whose difference, plus one, is exactly the width that counts the gaps between them. To keep the indices from overflowing on deep trees, subtract the level's leftmost index from every node's index (re-basing each level to start at 0).

Number Nodes as if the Tree Were Complete

BFS level by level, pairing each node with an index (root = 0; children $2i + 1, 2i + 2$). For each level, width = (last index – first index + 1); track the maximum. Re-base each level's indices by subtracting the first node's index to prevent overflow.

Algorithm

1. Queue holds $(node, index)$; push $(root, 0)$.
2. For each level: let *first* be the index of the level's first node. For every node, push $(left, 2 \cdot (i - first) + 1)$ and $(right, 2 \cdot (i - first) + 2)$. Width = last index – first index + 1; update the maximum.

Dry run

Dry Run. Tree: root 1; left 2 (left child 4); right 3 (right child 5).

- Level 0: (1, 0). Width 1.
- Level 1: (2, 0), (3, 1) (re-based). Width $1 - 0 + 1 = 2$.
- Level 2: from 2 push (4, 1); from 3 push (5, 4). First index 1; re-base to (4, 0), (5, 3). Width $3 - 0 + 1 = 4$.
- Maximum width = 4.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 int widthOfBinaryTree(TreeNode* root) {
12     if (root == nullptr) return 0;
13     long long best = 0;
14     queue<pair<TreeNode*, long long>> q;
15     q.push({root, 0});
16     while (!q.empty()) {
17         int s = q.size();
18         long long first = q.front().second, last = first;

```

```

19     for (int i = 0; i < s; i++) {
20         auto [node, idx] = q.front();
21         q.pop();
22         idx -= first; // re-base to avoid overflow
23         last = idx;
24         if (node->left) q.push({node->left, 2 * idx + 1});
25         if (node->right) q.push({node->right, 2 * idx + 2});
26     }
27     best = max(best, last + 1);
28 }
29 return (int)best;
30 }
31
32 int main() {
33     TreeNode* root = new TreeNode(1);
34     root->left = new TreeNode(2);
35     root->right = new TreeNode(3);
36     root->left->left = new TreeNode(4);
37     root->right->right = new TreeNode(5);
38
39     cout << "Max width: " << widthOfBinaryTree(root) << "\n";
40     return 0;
41 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one BFS pass. **Space Complexity.** $O(n)$ for the queue.

Key Takeaway

Positional indexing ($2i + 1$, $2i + 2$) is the standard way to reason about a tree's geometry when empty slots matter. Re-basing per level is the practical detail that keeps those indices from overflowing on a deep, sparse tree.

1.24 Children Sum Property

Problem Statement. The children-sum property holds if, for every node with at least one child, the node's value equals the sum of its children's values (a missing child counts as 0). Given a tree, *modify node values* (you may only increase them) so the property holds, without changing the tree's structure.

Pattern Recognition

This is not a check but a **construction**, and the “you may only increase values” rule forces a specific two-pass shape: push required values *down* on the way in (raise children up to at least the parent), then fix parents *up* on the way out (set each parent to its children's actual sum). The down-then-up structure is the key insight.

Understanding the problem carefully

If we only tried to fix parents to equal their children on the way up, a parent larger than its children's total would have no legal move (we cannot decrease). So first, descending, we *raise* the children: give each child at least the parent's value (bump the larger child, or either, up to the parent). Now the children's total is $\geq$ the parent, so on the way back up we can safely set each parent to the exact sum of its (possibly raised) children.

Push Down, Then Fix Up

Descent: at each node, compute the sum of its children's current values; if that sum is $\geq$ the node, set the node to the sum, else push the node's value down into (one of) its children. Recurse. **Ascent:** after both children are finalised, set the node to the sum of its children's values. Leaves are left untouched.

Algorithm

1. `FIX(node)`:

- (a) If `node` is null or a leaf, return.
- (b) Let $childSum = (\text{left val if any}) + (\text{right val if any})$. If $childSum \geq node.val$, set $node.val = childSum$; else copy $node.val$ into an existing child.
- (c) Recurse into left and right.
- (d) Set $node.val = (\text{left val if any}) + (\text{right val if any})$.

Dry run

Dry Run. Tree: root 50; children 7 and 2.

- Descent at 50: $childSum = 7 + 2 = 9 < 50$, so push 50 down: set left child to 50 (now 50, 2).
- Recurse into leaves 50 and 2: nothing to do.
- Ascent at root: set $root = 50 + 2 = 52$.
- Final: root 52, children 50, 2; $50 + 2 = 52$. Property holds.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 void fix(TreeNode* node) {
12     if (node == nullptr) return;
13     if (node->left == nullptr && node->right == nullptr) return;
14
15     int childSum = 0;
16     if (node->left) childSum += node->left->val;
17     if (node->right) childSum += node->right->val;
18
19     if (childSum >= node->val) {
20         node->val = childSum;           // children already suffice
21     } else {                          // push node value down
22         if (node->left) node->left->val = node->val;
23         else           node->right->val = node->val;
24     }

```

```

25     fix(node->left);
26     fix(node->right);
27
28
29     int total = 0;                                // fix up from actual
30     children
31     if (node->left) total += node->left->val;
32     if (node->right) total += node->right->val;
33     node->val = total;
34 }
35
36 int main() {
37     TreeNode* root = new TreeNode(50);
38     root->left = new TreeNode(7);
39     root->right = new TreeNode(2);
40
41     fix(root);
42     cout << root->val << " -> "
43         << root->left->val << ", " << root->right->val << "\n";
44     return 0;
45 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is handled once on the way down and once on the way up. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

When a constraint restricts *how* you may change values (here, only increases), a single pass rarely suffices – you pre-condition on the way down so the fix-up on the way back is always legal. Down-then-up is a recurring shape for tree mutations under monotone constraints.

1.25 All Nodes at Distance K from a Target

Problem Statement. Given the root, a target node, and an integer K , return the values of all nodes that are exactly K edges away from the target. Distance may go upward (toward ancestors) as well as downward.

Pattern Recognition

The twist is “*distance can go upward.*” A binary tree node has no parent pointer, so radiating outward in all directions is impossible with the raw structure. The fix: **build a parent map** so every node effectively becomes a graph vertex with up-to-three neighbours (left child, right child, parent), then run a plain **BFS** from the target.

Understanding the problem carefully

“All nodes at distance K ” is a BFS problem – but BFS needs to move in every direction, including up to the parent. So we first walk the tree once recording each node’s parent in a hash map. Now the tree behaves like an undirected graph: from any node we can step to its left child, right child, or parent. A breadth-first expansion from the target, stopping after K layers, lands exactly on the distance- K nodes.

Turn the Tree into a Graph, Then BFS K Layers

Record parents with one DFS. Start BFS from the target with a *visited* set (so we never walk back the way we came). Expand layer by layer; after K layers the queue holds precisely the nodes at distance K .

Algorithm

1. DFS to fill $parent[node]$ for every node.
2. BFS from target: mark it visited, distance 0.
3. Repeat K times: expand the current frontier to all unvisited neighbours (left, right, parent), marking them visited.
4. After K expansions, the frontier is the answer.

Dry run

Dry Run. Tree: root 3; left 5 (children 6, 2); right 1. Target 5, $K = 2$.

- Parents: $5 \rightarrow 3$, $6 \rightarrow 5$, $2 \rightarrow 5$, $1 \rightarrow 3$.
- Layer 0: $\{5\}$. Layer 1: neighbours of 5 = 6, 2, 3 (children and parent).
- Layer 2: from 6 nothing new; from 2 nothing new; from 3 neighbours are 5 (visited) and 1 (new). Frontier = $\{1\}$.
- Nodes at distance 2: 1 (and, in the classic example, 7, 4 if present under 1).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 void markParents(TreeNode* node, unordered_map<TreeNode*, TreeNode*>&
12     par) {
13     if (node == nullptr) return;
14     if (node->left) { par[node->left] = node; markParents(node->left,
15         par); }
16     if (node->right) { par[node->right] = node; markParents(node->right,
17         par); }
18 }
19
20 vector<int> distanceK(TreeNode* root, TreeNode* target, int K) {
21     unordered_map<TreeNode*, TreeNode*> par;
22     markParents(root, par);
23
24     unordered_set<TreeNode*> visited;
25     queue<TreeNode*> q;

```

```

23     q.push(target);
24     visited.insert(target);
25
26     int dist = 0;
27     while (!q.empty()) {
28         if (dist == K) break;
29         int s = q.size();
30         for (int i = 0; i < s; i++) {
31             TreeNode* node = q.front(); q.pop();
32             for (TreeNode* nb : {node->left, node->right, par.count(
33                 node) ? par[node] : nullptr}) {
34                 if (nb && !visited.count(nb)) { visited.insert(nb); q.
35                     push(nb); }
36             }
37             dist++;
38         }
39         vector<int> ans;
40         while (!q.empty()) { ans.push_back(q.front()->val); q.pop(); }
41         return ans;
42     }
43
44     int main() {
45         TreeNode* root = new TreeNode(3);
46         root->left = new TreeNode(5);
47         root->right = new TreeNode(1);
48         root->left->left = new TreeNode(6);
49         root->left->right = new TreeNode(2);
50
51         vector<int> res = distanceK(root, root->left, 2);
52         sort(res.begin(), res.end());
53         for (int x : res) cout << x << " ";
54         cout << "\n";
55         return 0;
56     }

```

Complexity Analysis

Time Complexity. $O(n)$ – one DFS to record parents and a BFS visiting each node at most once. **Space Complexity.** $O(n)$ for the parent map, visited set, and queue.

Key Takeaway

“Distance in all directions” on a tree is a graph-BFS problem in disguise; the enabling move is manufacturing parent pointers so upward travel becomes possible. This parent-map trick recurs in the next section’s burning-tree problem.

1.26 Minimum Time to Burn the Tree

Problem Statement. A fire starts at a given target node. Each second, the fire spreads from every burning node to its adjacent nodes (children and parent). Return the minimum number of seconds needed to burn the entire tree.

Pattern Recognition

This is the “distance in all directions” idea taken to its conclusion: the answer is the **maximum distance** from the target to *any* node, because the last node to catch fire is the farthest one. Same recipe – **parent map plus BFS from the target** – but instead of stopping at layer K , count how many layers it takes to exhaust the tree.

Understanding the problem carefully

Fire spreads one edge per second in every direction. A node at graph distance d from the source catches fire at second d . The whole tree is ablaze once the farthest node burns, so the time equals the eccentricity of the target: the greatest distance to any node. A BFS from the target, counting completed layers until the queue empties, measures exactly this.

Answer = Number of BFS Layers to Empty the Tree

Build the parent map, BFS from the target with a visited set. Each fully processed frontier is one second of spread. The number of layers after the *initial* one (the source itself, at time 0) is the burn time – i.e. increment a timer each time you expand a non-empty next frontier.

Algorithm

1. DFS to fill *parent*[node].
2. BFS from target; *time* = −1 (so the first layer, the source, corresponds to time 0).
3. Each layer: mark all unvisited neighbours; if the layer expanded anything, increment *time*.
Actually, increment *time* once per processed frontier.
4. Return *time*.

(Concretely: initialise *time* = 0, and increment it only when a frontier produces at least one newly burning node.)

Dry run

Dry Run. Tree: root 1; left 2 (left child 4); right 3. Start fire at 4.

- $t = 0$: {4} burning.
- $t = 1$: neighbour of 4 is 2. {2}.
- $t = 2$: neighbours of 2 are 1 (and 4 burnt). {1}.
- $t = 3$: neighbour of 1 is 3. {3}.
- Tree fully burnt at $t = 3$.

C++ implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
```

```

6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 void markParents(TreeNode* node, unordered_map<TreeNode*, TreeNode*>&
    par) {
12     if (node == nullptr) return;
13     if (node->left) { par[node->left] = node; markParents(node->left,
        par); }
14     if (node->right) { par[node->right] = node; markParents(node->right
        , par); }
15 }
16
17 int timeToBurn(TreeNode* root, TreeNode* target) {
18     unordered_map<TreeNode*, TreeNode*> par;
19     markParents(root, par);
20
21     unordered_set<TreeNode*> visited;
22     queue<TreeNode*> q;
23     q.push(target);
24     visited.insert(target);
25
26     int time = 0;
27     while (!q.empty()) {
28         int s = q.size();
29         bool spread = false;
30         for (int i = 0; i < s; i++) {
31             TreeNode* node = q.front(); q.pop();
32             for (TreeNode* nb : {node->left, node->right, par.count(
                node) ? par[node] : nullptr}) {
33                 if (nb && !visited.count(nb)) {
34                     visited.insert(nb);
35                     q.push(nb);
36                     spread = true;
37                 }
38             }
39         }
40         if (spread) time++;
41     }
42     return time;
43 }
44
45 int main() {
46     TreeNode* root = new TreeNode(1);
47     root->left = new TreeNode(2);
48     root->right = new TreeNode(3);
49     root->left->left = new TreeNode(4);
50
51     cout << "Burn time: " << timeToBurn(root, root->left->left) << "\n"
        ;
52     return 0;
53 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one DFS plus one BFS, each visiting every node once. **Space Complexity.** $O(n)$ for the parent map, visited set, and queue.

Key Takeaway

Burning the tree is “nodes at distance K ” with K pushed to the maximum: the answer is the number of BFS layers to consume the whole tree. Once you see fire-spread problems as level-counting BFS on the parent-augmented graph, they become routine.

1.27 Count Nodes in a Complete Binary Tree

Problem Statement. Given the root of a *complete* binary tree (every level full except possibly the last, which is filled left to right), count its nodes in better than $O(n)$ time.

Pattern Recognition

A plain traversal counts in $O(n)$, but the word *complete* is an invitation to do better. The key fact: if a subtree’s leftmost depth equals its rightmost depth, it is a **perfect** tree and its node count is $2^{\text{height}} - 1$ by formula – no traversal needed. Only along the one “ragged” edge do we recurse, giving $O(\log^2 n)$.

Understanding the problem carefully

Walk down the far left of a subtree to get its left height; walk down the far right to get its right height. If they are equal, every level is full, so the subtree is perfect and has exactly $2^h - 1$ nodes – return that instantly. If they differ, the last level is only partially filled, so recurse into both children and add 1 for the current node. Crucially, at most one child per level is ragged; the other is perfect and resolved by formula, so the recursion depth is $O(\log n)$ and each level costs $O(\log n)$ to measure the two heights.

Perfect Subtrees Close by Formula

COUNT(node): if null return 0. Measure left height l (all-left walk) and right height r (all-right walk). If $l = r$, return $2^l - 1$. Otherwise return $1 + \text{COUNT}(\text{left}) + \text{COUNT}(\text{right})$.

Algorithm

1. LEFTHEIGHT(node): count nodes going only left until null.
2. RIGHTEIGHT(node): count nodes going only right until null.
3. COUNT(node): if null return 0; if leftHeight = rightHeight = h return $2^h - 1$; else return $1 + \text{COUNT}(\text{left}) + \text{COUNT}(\text{right})$.

Dry run

Dry Run. Complete tree: root 1; children 2, 3; 2’s children 4, 5; 3’s left child 6.

- At 1: left height = 3 ($1 \rightarrow 2 \rightarrow 4$), right height = 2 ($1 \rightarrow 3$). Unequal $\Rightarrow$ recurse.
- Left subtree at 2: left height = right height = 2; perfect, $2^2 - 1 = 3$ nodes.

- Right subtree at 3: left height = 2, right height = 1; recurse $\rightarrow 1 + \text{COUNT}(6) + 0 = 1 + 1 = 2$.
- Total = $1 + 3 + 2 = 6$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 int leftHeight(TreeNode* n) {
12     int h = 0;
13     while (n) { h++; n = n->left; }
14     return h;
15 }
16 int rightHeight(TreeNode* n) {
17     int h = 0;
18     while (n) { h++; n = n->right; }
19     return h;
20 }
21
22 int countNodes(TreeNode* root) {
23     if (root == nullptr) return 0;
24     int lh = leftHeight(root), rh = rightHeight(root);
25     if (lh == rh) return (1 << lh) - 1; // perfect subtree
26     return 1 + countNodes(root->left) + countNodes(root->right);
27 }
28
29 int main() {
30     TreeNode* root = new TreeNode(1);
31     root->left = new TreeNode(2);
32     root->right = new TreeNode(3);
33     root->left->left = new TreeNode(4);
34     root->left->right = new TreeNode(5);
35     root->right->left = new TreeNode(6);
36
37     cout << "Node count: " << countNodes(root) << "\n";
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(\log^2 n)$ – the recursion has $O(\log n)$ depth, and each level spends $O(\log n)$ measuring the two heights. **Space Complexity.** $O(\log n)$ for the recursion stack.

Key Takeaway

When a structural guarantee (“complete”) is handed to you, look for a closed-form shortcut before defaulting to a full traversal. Resolving perfect subtrees by the $2^h - 1$ formula and recursing only along the ragged boundary is the canonical sub-linear tree count.

1.28 Construct a Binary Tree from Preorder and Inorder

Problem Statement. Given the preorder and inorder traversals of a binary tree with unique values, reconstruct and return the tree.

Pattern Recognition

“Rebuild a tree from two traversals” is a divide-and-conquer: **preorder’s first element is always the root**, and locating that root inside the inorder array **splits inorder into the left subtree and the right subtree**. Recurse on the two halves. A hash map from value to inorder index turns the “locate the root” step from $O(n)$ into $O(1)$.

Understanding the problem carefully

Preorder visits root, then all of left, then all of right – so its first value is the root. Inorder visits left, root, right – so once we know the root’s value, everything to its left in the inorder array is the left subtree and everything to its right is the right subtree. The count of left-subtree nodes (from inorder) tells us exactly how many of the following preorder elements belong to the left subtree, letting us slice preorder too. Recursing reconstructs both subtrees.

Preorder Gives the Root; Inorder Gives the Split

Take the next preorder value as the current root. Find it in inorder at position p : the p elements left of it form the left subtree, the rest form the right subtree. Consume preorder left-to-right (root, then build left, then build right). An index map makes finding p instant.

Algorithm

1. Build pos : value $\rightarrow$ its index in inorder.
2. BUILD($inLo$, $inHi$) using a shared preorder cursor $preIdx = 0$:
 - (a) If $inLo > inHi$, return null.
 - (b) $rootVal = preorder[preIdx++]$; make a node.
 - (c) $p = pos[rootVal]$.
 - (d) $node.left = BUILD(inLo, p - 1)$; $node.right = BUILD(p + 1, inHi)$.
 - (e) Return node.

Dry run

Dry Run. preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7].

- Root = 3 (preorder[0]); in inorder at index 1: left = [9], right = [15, 20, 7].
- Left: root = 9; inorder slice [9] has no children – leaf.
- Right: root = 20 (next preorder); in inorder index 3: left = [15], right = [7]; both leaves.
- Reconstructed: 3 with left 9 and right 20(15, 7).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 int preIdx;
12
13 TreeNode* build(vector<int>& preorder, int inLo, int inHi,
14                unordered_map<int,int>& pos) {
15     if (inLo > inHi) return nullptr;
16     int rootVal = preorder[preIdx++];
17     TreeNode* node = new TreeNode(rootVal);
18     int p = pos[rootVal];
19     node->left = build(preorder, inLo, p - 1, pos);
20     node->right = build(preorder, p + 1, inHi, pos);
21     return node;
22 }
23
24 TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
25     unordered_map<int,int> pos;
26     for (int i = 0; i < (int)inorder.size(); i++) pos[inorder[i]] = i;
27     preIdx = 0;
28     return build(preorder, 0, inorder.size() - 1, pos);
29 }
30
31 void inorderPrint(TreeNode* n) {
32     if (!n) return;
33     inorderPrint(n->left);
34     cout << n->val << " ";
35     inorderPrint(n->right);
36 }
37
38 int main() {
39     vector<int> preorder = {3, 9, 20, 15, 7};
40     vector<int> inorder = {9, 3, 15, 20, 7};
41     TreeNode* root = buildTree(preorder, inorder);
42     inorderPrint(root);           // should reprint the inorder
43     cout << "\n";
44     return 0;
45 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is created once and the inorder position is an $O(1)$ map lookup. **Space Complexity.** $O(n)$ for the map, plus $O(h)$ recursion.

Key Takeaway

Reconstruction hinges on two facts: one traversal names the root, another splits the rest into left and right. The shared preorder cursor consumed left-to-right is the clean way to

avoid slicing arrays at every recursive call.

1.29 Construct a Binary Tree from Postorder and Inorder

Problem Statement. Given the postorder and inorder traversals of a binary tree with unique values, reconstruct and return the tree.

Pattern Recognition

This is the previous section mirrored. Postorder visits left, right, *root* – so the root is the **last** element, not the first. Read postorder *right to left* and it behaves like preorder with the children swapped: root, then right subtree, then left subtree. So build the **right subtree before the left**, consuming postorder from the back.

Understanding the problem carefully

Because postorder ends with the root, a cursor moving backward through postorder yields roots in the order root, right-subtree-root, Inorder still splits into left and right around the root's position. The one change from the preorder version: since we now encounter the right subtree's root immediately after the current root (reading backward), we must recurse *right first*, then left, so the backward cursor lines up with the subtrees correctly.

Postorder from the Back Names Roots; Build Right, Then Left

Take the current root as `postorder[postIdx]` and decrement `postIdx`. Split inorder at the root's position. Recurse into the *right* subtree first (`node.right`), then the *left* (`node.left`) – the reverse of the preorder construction.

Algorithm

1. Build `pos`: value $\rightarrow$ inorder index. Set `postIdx` = $n - 1$.
2. `BUILD(inLo, inHi)`:
 - (a) If `inLo > inHi`, return null.
 - (b) `rootVal` = `postorder[postIdx]`; make node.
 - (c) `p` = `pos[rootVal]`.
 - (d) `node.right` = `BUILD(p + 1, inHi)`; `node.left` = `BUILD(inLo, p - 1)`.
 - (e) Return node.

Dry run

Dry Run. `postorder` = [9, 15, 7, 20, 3], `inorder` = [9, 3, 15, 20, 7].

- Root = 3 (postorder last); inorder index 1: left = [9], right = [15, 20, 7].
- Build right first: next root (moving back) = 20; inorder index 3: right = [7] (leaf), left = [15] (leaf).
- Build left: next root = 9 – leaf.
- Reconstructed tree matches the original.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 int postIdx;
12
13 TreeNode* build(vector<int>& postorder, int inLo, int inHi,
14                unordered_map<int,int>& pos) {
15     if (inLo > inHi) return nullptr;
16     int rootVal = postorder[postIdx--];
17     TreeNode* node = new TreeNode(rootVal);
18     int p = pos[rootVal];
19     node->right = build(postorder, p + 1, inHi, pos); // right first
20     node->left = build(postorder, inLo, p - 1, pos);
21     return node;
22 }
23
24 TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
25     unordered_map<int,int> pos;
26     for (int i = 0; i < (int)inorder.size(); i++) pos[inorder[i]] = i;
27     postIdx = postorder.size() - 1;
28     return build(postorder, 0, inorder.size() - 1, pos);
29 }
30
31 void inorderPrint(TreeNode* n) {
32     if (!n) return;
33     inorderPrint(n->left);
34     cout << n->val << " ";
35     inorderPrint(n->right);
36 }
37
38 int main() {
39     vector<int> inorder = {9, 3, 15, 20, 7};
40     vector<int> postorder = {9, 15, 7, 20, 3};
41     TreeNode* root = buildTree(inorder, postorder);
42     inorderPrint(root);
43     cout << "\n";
44     return 0;
45 }

```

Complexity Analysis

Time Complexity. $O(n)$ with the inorder index map. **Space Complexity.** $O(n)$ for the map plus $O(h)$ recursion.

Key Takeaway

Postorder+inorder is preorder+inorder read backward with the child order flipped. Seeing the symmetry means you derive one from the other instead of memorising two separate

algorithms. Note: preorder+postorder alone cannot uniquely rebuild a tree – inorder is what fixes the left/right split.

1.30 Serialize and Deserialize a Binary Tree

Problem Statement. Design two functions: *serialize*, which encodes a binary tree into a single string, and *deserialize*, which rebuilds the exact tree from that string. Values may repeat and the tree may be arbitrary.

Pattern Recognition

Unlike reconstruction from two traversals, here we control the format – so we choose one that is *self-describing* by **recording null children explicitly**. With null markers, a *single* preorder walk carries all the structure needed to rebuild the tree unambiguously. The design freedom is the whole point.

Understanding the problem carefully

Two traversals are needed to rebuild a tree only because a normal traversal hides where subtrees end. If we instead emit a special token (say #) whenever we hit a null child, the string records the tree’s exact shape. Reading that string back in the same preorder order, a null token means “this child is empty,” so deserialization mirrors serialization step for step – read a token, build a node (or return null), then recursively build its left and right children.

Preorder with Explicit Null Markers Is Self-Sufficient

Serialize: preorder traversal; output each value, and output a sentinel # for every null child, separated by commas. **Deserialize:** read tokens in order; # yields null, otherwise create a node and recursively deserialize its left then right child. The null markers tell the reader exactly where each subtree stops.

Algorithm

1. **SERIALIZE**(node, out): if node is null, append #,; else append node.val, then SERIALIZE(left) and SERIALIZE(right).
2. **DESERIALIZE**: read the next token; if # return null; else make a node, set node.left = DESERIALIZE, node.right = DESERIALIZE; return node.

Dry run

Dry Run. Tree: root 1; left 2; right 3 (with children 4,5).

- Serialize: 1,2,#,#,3,4,#,#,5,#,#, – 2 is a leaf (two #), then 3, then its children 4,5.
- Deserialize reads 1 (node), builds left from 2 (then two # → leaf), builds right from 3, whose children come from 4 and 5.
- The rebuilt tree is identical to the original.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 void serialize(TreeNode* node, string& out) {
12     if (node == nullptr) { out += "#,"; return; }
13     out += to_string(node->val) + ",";
14     serialize(node->left, out);
15     serialize(node->right, out);
16 }
17
18 TreeNode* deserialize(istringstream& in) {
19     string token;
20     getline(in, token, ',');
21     if (token == "#") return nullptr;
22     TreeNode* node = new TreeNode(stoi(token));
23     node->left = deserialize(in);
24     node->right = deserialize(in);
25     return node;
26 }
27
28 void inorderPrint(TreeNode* n) {
29     if (!n) return;
30     inorderPrint(n->left);
31     cout << n->val << " ";
32     inorderPrint(n->right);
33 }
34
35 int main() {
36     TreeNode* root = new TreeNode(1);
37     root->left = new TreeNode(2);
38     root->right = new TreeNode(3);
39     root->right->left = new TreeNode(4);
40     root->right->right = new TreeNode(5);
41
42     string s;
43     serialize(root, s);
44     cout << "Serialized: " << s << "\n";
45
46     istringstream in(s);
47     TreeNode* rebuilt = deserialize(in);
48     cout << "Inorder of rebuilt: ";
49     inorderPrint(rebuilt);
50     cout << "\n";
51     return 0;
52 }

```

Complexity Analysis

Time Complexity. $O(n)$ for both serialize and deserialize – each node and each null marker is written and read once. **Space Complexity.** $O(n)$ for the string, plus $O(h)$ recursion.

Key Takeaway

When you own the encoding, adding explicit null markers makes a single traversal invertible – no second traversal required. This “sentinel for absence” idea is the general trick for making any recursive structure round-trip through a flat string.

1.31 Morris Inorder Traversal

Problem Statement. Return the inorder traversal of a binary tree using $O(1)$ extra space – no recursion and no explicit stack.

Pattern Recognition

“ $O(1)$ space,” “no stack,” “no recursion” points to **Morris traversal**, which recovers the “go back up to the ancestor” capability – normally provided by a stack – by temporarily rewiring the tree’s own unused null pointers into **threads** back to the ancestor, then undoing them.

Understanding the problem carefully

The only reason inorder needs a stack is to remember, after finishing a left subtree, which node to return to. Morris’s insight: the node we must return to is exactly the *inorder predecessor*’s successor – and the inorder predecessor of a node (the rightmost node of its left subtree) has a null right pointer we can borrow. We point that null back at the current node, creating a temporary “thread.” After the left subtree is walked, we arrive back via the thread, record the node, remove the thread (restoring the tree), and move right.

Thread the Predecessor’s Null Right Pointer Back to You

At each node with a left child, find its inorder predecessor (go left once, then all the way right). If the predecessor’s right is null, set it to the current node (make the thread) and move left. If it is already the current node, the left subtree is done: remove the thread, record the node, and move right. Nodes without a left child are recorded immediately and you move right.

Algorithm

1. $curr = \text{root}$.
2. While $curr$ is non-null:
 - (a) If $curr$ has no left child: record $curr$; move $curr = curr.\text{right}$.
 - (b) Else find $pred = \text{rightmost node of } curr.\text{left}$ (not passing through an existing thread):
 - i. If $pred.\text{right}$ is null: set it to $curr$; move $curr = curr.\text{left}$.
 - ii. Else ($pred.\text{right} = curr$): reset $pred.\text{right}$ to null; record $curr$; move $curr = curr.\text{right}$.

Dry run

Dry Run. Tree: root 2; left 1; right 3.

- $curr=2$, has left 1. Predecessor of 2 is 1; 1.right null $\rightarrow$ thread $1 \rightarrow 2$; move left to 1.
- $curr=1$, no left $\rightarrow$ record 1; move right via thread to 2.
- $curr=2$, has left; predecessor 1.right = 2 (thread) $\rightarrow$ remove thread, record 2, move right to 3.
- $curr=3$, no left $\rightarrow$ record 3.
- Inorder: 1, 2, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 vector<int> morrisInorder(TreeNode* root) {
12     vector<int> out;
13     TreeNode* curr = root;
14     while (curr != nullptr) {
15         if (curr->left == nullptr) {
16             out.push_back(curr->val);           // no left: record, go
17             curr = curr->right;                  right
18         } else {
19             TreeNode* pred = curr->left;
20             while (pred->right != nullptr && pred->right != curr)
21                 pred = pred->right;             // find inorder
22             if (pred->right == nullptr) {        predecessor
23                 pred->right = curr;             // create thread
24                 curr = curr->left;
25             } else {
26                 pred->right = nullptr;          // remove thread
27                 out.push_back(curr->val);        // left subtree done:
28                 curr = curr->right;              record
29             }
30         }
31     }
32     return out;
33 }
34
35 int main() {
36     TreeNode* root = new TreeNode(2);
37     root->left = new TreeNode(1);
38     root->right = new TreeNode(3);

```

```

39
40     for (int x : morrisInorder(root)) cout << x << " ";
41     cout << "\n";
42     return 0;
43 }

```

Complexity Analysis

Time Complexity. $O(n)$ – although finding predecessors seems to add work, each edge is traversed at most a constant number of times, so the total remains linear. **Space Complexity.** $O(1)$ – no stack or recursion; only a few pointers, with the tree restored to its original shape at the end.

Key Takeaway

Morris trades the stack for temporary threads woven from the tree’s own null pointers, achieving true $O(1)$ space. The discipline that keeps it correct is always removing a thread the second time you meet it, so the tree is left exactly as it began.

1.32 Morris Preorder Traversal

Problem Statement. Return the preorder traversal of a binary tree in $O(1)$ extra space – no recursion and no explicit stack.

Pattern Recognition

Morris preorder is Morris inorder with the “record” step moved. Recall from Section 1.2 that preorder and inorder are the same walk with the node recorded at a different moment. In the threaded walk, that means recording the node *when you create the thread* (first visit) rather than when you remove it.

Understanding the problem carefully

The Morris machinery – thread the predecessor’s null right pointer, walk left, later return and unthread – visits each node twice when it has a left child: once on the way in (creating the thread) and once on the way back (removing it). Inorder records on the second visit; preorder must record on the *first*. So we output the node’s value at the moment we create the thread (and immediately for nodes with no left child), which places the node before its subtrees – exactly preorder.

Same Threads, Record on the First Visit

Identical to Morris inorder, with one line moved: when you create a thread (first time you reach a node with a left child), record the node *then*, before moving left. Nodes without a left child are still recorded immediately. When you later remove a thread, do *not* record again.

Algorithm

1. $curr = root$.
2. While $curr$ is non-null:
 - (a) If $curr$ has no left child: record $curr$; move right.

- (b) Else find predecessor *pred* (rightmost of the left subtree, not through a thread):
- i. If *pred.right* is null: **record *curr*** (first visit); set *pred.right* = *curr*; move left.
 - ii. Else: reset *pred.right* to null; move right (no recording).

Dry run

Dry Run. Tree: root 1; left 2; right 3.

- *curr* = 1, has left. Predecessor 2.right null → record 1; thread 2 → 1; move left to 2.
- *curr* = 2, no left → record 2; move right via thread to 1.
- *curr* = 1, predecessor 2.right = 1 (thread) → remove thread; move right to 3 (no record).
- *curr* = 3, no left → record 3.
- Preorder: 1, 2, 3.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 vector<int> morrisPreorder(TreeNode* root) {
12     vector<int> out;
13     TreeNode* curr = root;
14     while (curr != nullptr) {
15         if (curr->left == nullptr) {
16             out.push_back(curr->val);
17             curr = curr->right;
18         } else {
19             TreeNode* pred = curr->left;
20             while (pred->right != nullptr && pred->right != curr)
21                 pred = pred->right;
22             if (pred->right == nullptr) {
23                 out.push_back(curr->val);    // record on FIRST visit
24                 pred->right = curr;
25                 curr = curr->left;
26             } else {
27                 pred->right = nullptr;    // second visit: just
28                 unthread                  // remove thread
29                 curr = curr->right;
30             }
31         }
32     }
33     return out;
34 }
35
36 int main() {
37     TreeNode* root = new TreeNode(1);

```

```

37     root->left = new TreeNode(2);
38     root->right = new TreeNode(3);
39
40     for (int x : morrisPreorder(root)) cout << x << " ";
41     cout << "\n";
42     return 0;
43 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each edge is walked a constant number of times. **Space Complexity.** $O(1)$ – threads only, tree restored at the end.

Key Takeaway

Once Morris inorder is understood, preorder is a one-line change – record on the first visit instead of the second – reaffirming the chapter’s opening theme that the three depth-first orders differ only in *when* the node is recorded, threading or not.

1.33 Flatten a Binary Tree to a Linked List

Problem Statement. Flatten a binary tree in place so that it becomes a “linked list”: every node’s left child is null and its right child points to the next node in *preorder*. Use the existing right pointers as the list’s next pointers.

Pattern Recognition

The target order is *preorder*, so the flattened list is node, then all of the (flattened) left subtree, then all of the (flattened) right subtree. The elegant $O(1)$ -space solution is a **reverse-preorder** (right, left, node) walk that stitches each node’s right pointer to the previously visited node – building the list backwards.

Understanding the problem carefully

In the final list, after any node come all its left-subtree nodes, then its right-subtree nodes. If we process nodes in the *reverse* of preorder – that is right subtree, then left subtree, then the node – then at the moment we handle a node, the entire portion of the list that should come *after* it has already been assembled and is pointed to by a running *prev* pointer. So we set `node.right = prev`, `node.left = null`, and update `prev = node`.

Build the List Backwards with a Running *prev*

Traverse in right, left, node order. Keep *prev*, the head of the already-flattened suffix. At each node: set its right pointer to *prev*, its left pointer to null, then set *prev* to this node. When the traversal finishes, *prev* is the root of the fully flattened list.

Algorithm

1. *prev* = null.
2. FLATTEN(node):
 - (a) If node is null, return.
 - (b) FLATTEN(node.right).

(c) FLATTEN(node.left).

(d) node.right = prev; node.left = null; prev = node.

Dry run

Dry Run. Tree: root 1; left 2 (children 3,4); right 5 (right child 6). Preorder: 1, 2, 3, 4, 5, 6.

- Reverse-preorder processes 6 first: 6.right=null, prev= 6.
- Then 5: 5.right= 6, prev= 5. Then 4: 4.right= 5, prev= 4. Then 3: 3.right= 4. Then 2: 2.right= 3, left null. Then 1: 1.right= 2, left null.
- Result chain via right pointers: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$, all left pointers null.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11  TreeNode* prevNode = nullptr;
12
13  void flatten(TreeNode* node) {
14      if (node == nullptr) return;
15      flatten(node->right);    // right first
16      flatten(node->left);     // then left
17      node->right = prevNode;  // stitch onto the already-built suffix
18      node->left = nullptr;
19      prevNode = node;
20  }
21
22  int main() {
23      TreeNode* root = new TreeNode(1);
24      root->left = new TreeNode(2);
25      root->right = new TreeNode(5);
26      root->left->left = new TreeNode(3);
27      root->left->right = new TreeNode(4);
28      root->right->right = new TreeNode(6);
29
30      prevNode = nullptr;
31      flatten(root);
32
33      for (TreeNode* c = root; c != nullptr; c = c->right)
34          cout << c->val << " ";
35      cout << "\n";
36      return 0;
37  }

```

Complexity Analysis

Time Complexity. $O(n)$ – each node is visited once. **Space Complexity.** $O(h)$ for the recursion stack; a Morris-style variant achieves $O(1)$ by rewiring the rightmost node of each left subtree to the right subtree, in the spirit of the previous two sections.

Key Takeaway

Processing nodes in reverse of the target order, while carrying a *prev* pointer to the already-built tail, is a recurring in-place linking technique – the same idea that reverses a linked list. Flattening a tree is that trick applied along a preorder spine.